

Azure SOC Automation Lab — *Cloud-Native Threat Detection & Automated Response with Microsoft Sentinel*

Microsoft Sentinel Home Lab

Azure Cloud SIEM Implementation and Configuration

Executive Summary

This report documents the complete configuration and deployment of a Microsoft Sentinel Security Information and Event Management (SIEM) home laboratory environment on Microsoft Azure. The lab demonstrates practical implementation of cloud-based security monitoring, log analytics integration, and security event correlation capabilities. This environment serves as a functional demonstration of enterprise-level security operations center (SOC) infrastructure in a controlled cloud setting.

Project Objectives

The primary objectives of this laboratory implementation include:

- Deploy and configure a complete Azure infrastructure for security monitoring
- Implement Microsoft Sentinel for centralized log collection and analysis
- Configure intentionally vulnerable network security settings for penetration testing
- Establish Log Analytics workspace for event aggregation and KQL query execution
- Create data collection rules for Windows Security Events

Laboratory Architecture Overview

The laboratory environment consists of the following core components:

Component	Configuration Details
Resource Group	MYI-SOC-lab (France Central region)
Virtual Network	VNET-SOC-lab (10.0.0.0/16 address space)
Virtual Machine	CORP-NET-EAST (Windows Server 2022 Datacenter, Standard E2s v3)
Log Analytics	LAW-SOC-lab workspace (Pay-as-you-go pricing tier)
SIEM Platform	Microsoft Sentinel with Windows Security Events connector

Detailed Implementation Steps

Step 1: Azure Resource Group Creation

The initial phase of the laboratory deployment involved creating a dedicated Azure Resource Group to contain all security monitoring infrastructure components. The resource group, named MYI-SOC-lab, was deployed in the France Central region to optimize latency and ensure compliance with European data residency requirements.

Figure 1: Resource Group Overview - MYI-SOC-lab

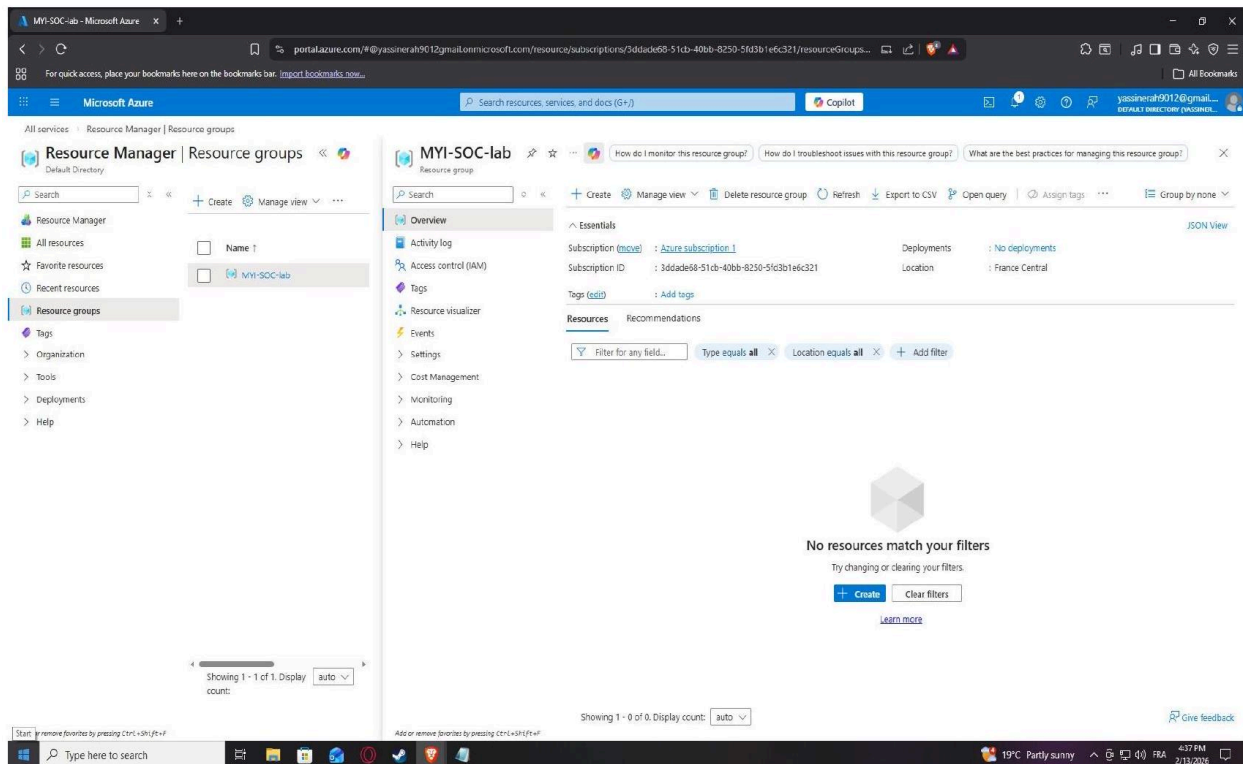


Figure 1 demonstrates the Azure Portal view of the MYI-SOC-lab resource group immediately following creation. The screenshot shows the empty resource group state with no deployments recorded.

Step 2: Virtual Network Configuration

A dedicated virtual network was provisioned to provide network isolation and connectivity for the laboratory environment. The virtual network, designated VNET-SOC-lab, implements a standard RFC 1918 private address space of 10.0.0.0/16, providing 65,536 available IP addresses for future expansion.

Figure 2: Virtual Network Creation - Review and Create Phase

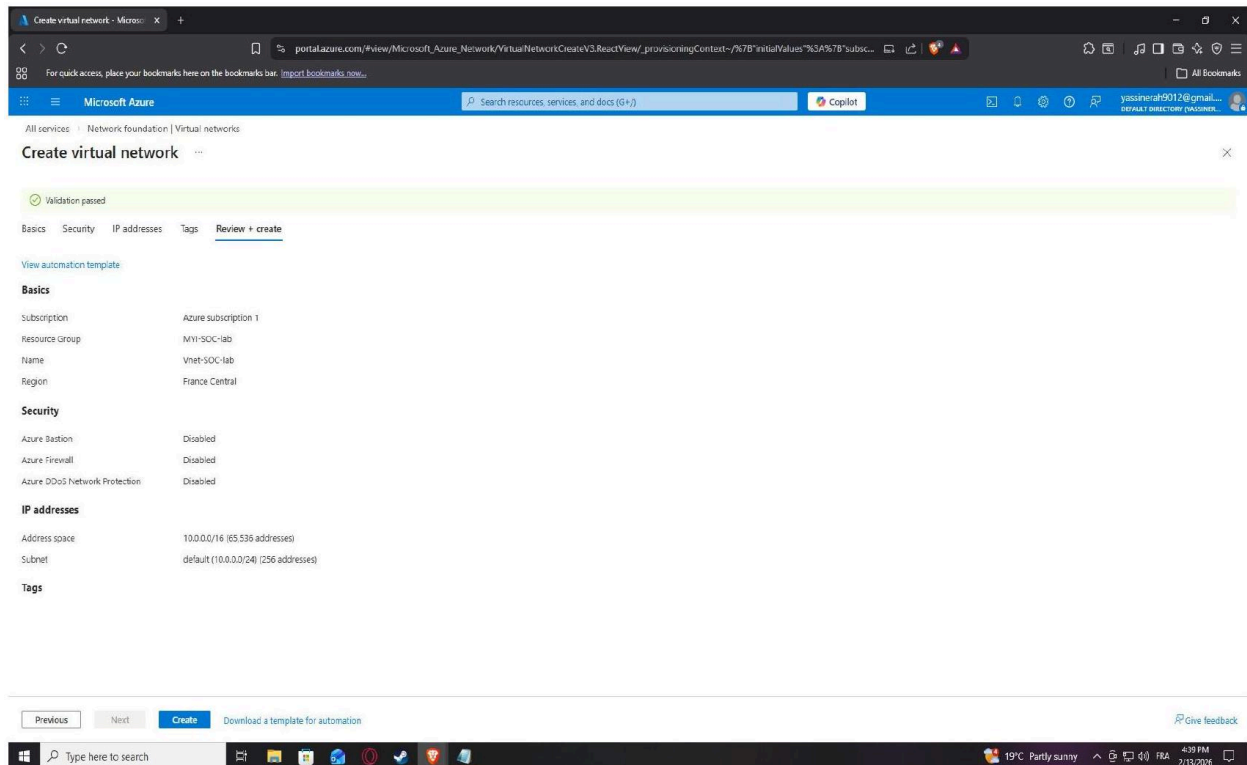


Figure 3: Virtual Machine Configuration - Basics Tab

Figure 3 displays the final validation screen prior to virtual network deployment. Critical configurations include:

- Address space allocation: 10.0.0.0/16 (65,536 addresses)
- Default subnet configuration: 10.0.0.0/24 (256 addresses)
- Security features disabled: Azure Bastion, Azure Firewall, and Azure DDoS Network Protection intentionally disabled for cost optimization in laboratory environment

Step 3: Windows Server Virtual Machine Deployment

A Windows Server 2022 Datacenter virtual machine was deployed to serve as the primary monitored endpoint for security event generation and collection. The virtual machine configuration was intentionally designed with reduced security controls to facilitate security testing and event generation.

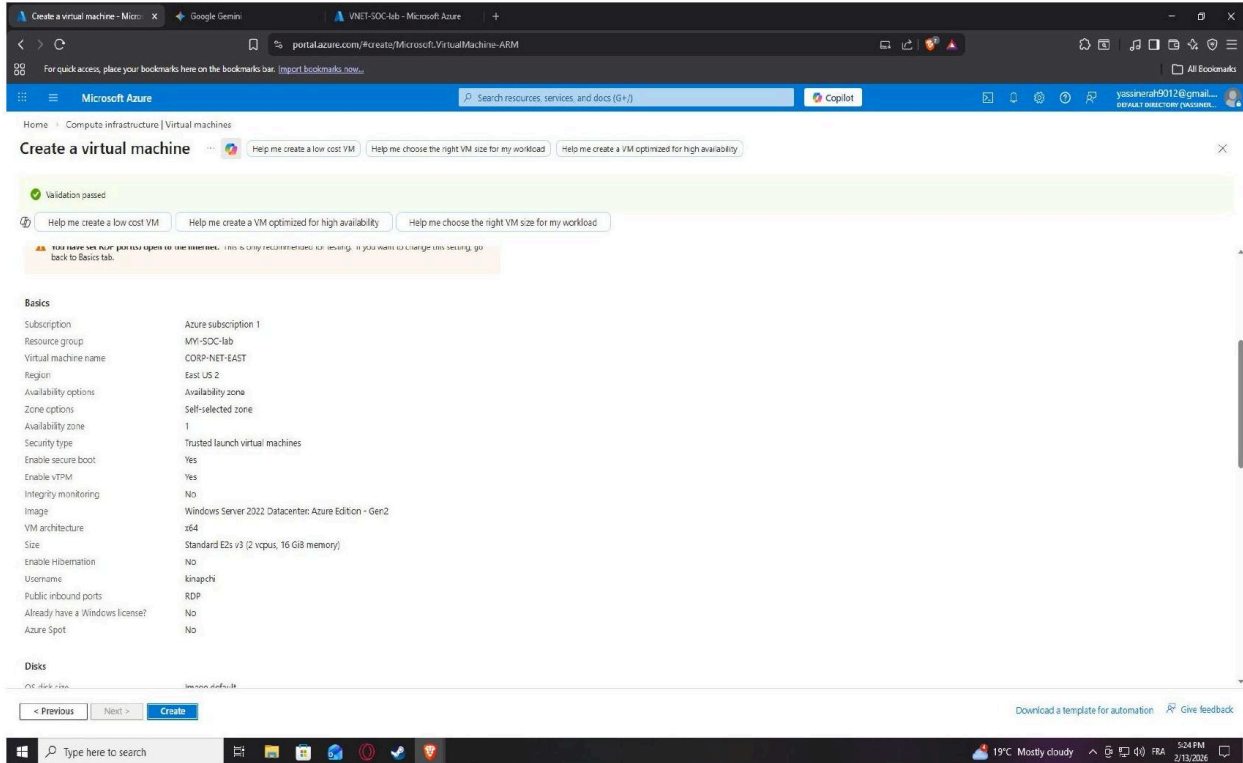


Figure 4: Virtual Machine Configuration - Disk and Networking

Figure 4 illustrates the fundamental virtual machine configuration parameters:

- Resource allocation: Deployed to MYI-SOC-lab resource group
- Instance naming: CORP-NET-EAST (simulating corporate network infrastructure)
- Geographic placement: East US 2 region (distributed across regions from resource group)
- Availability configuration: Availability zone deployment with self-selected zone
- Security features: Trusted launch enabled with vTPM and Secure Boot
- Operating system: Windows Server 2022 Datacenter - Azure Edition - Gen2
- Compute capacity: Standard E2s v3 (2 vCPUs, 16 GiB memory)
- Administrative access: Username configured as 'kinapchi', RDP port opened

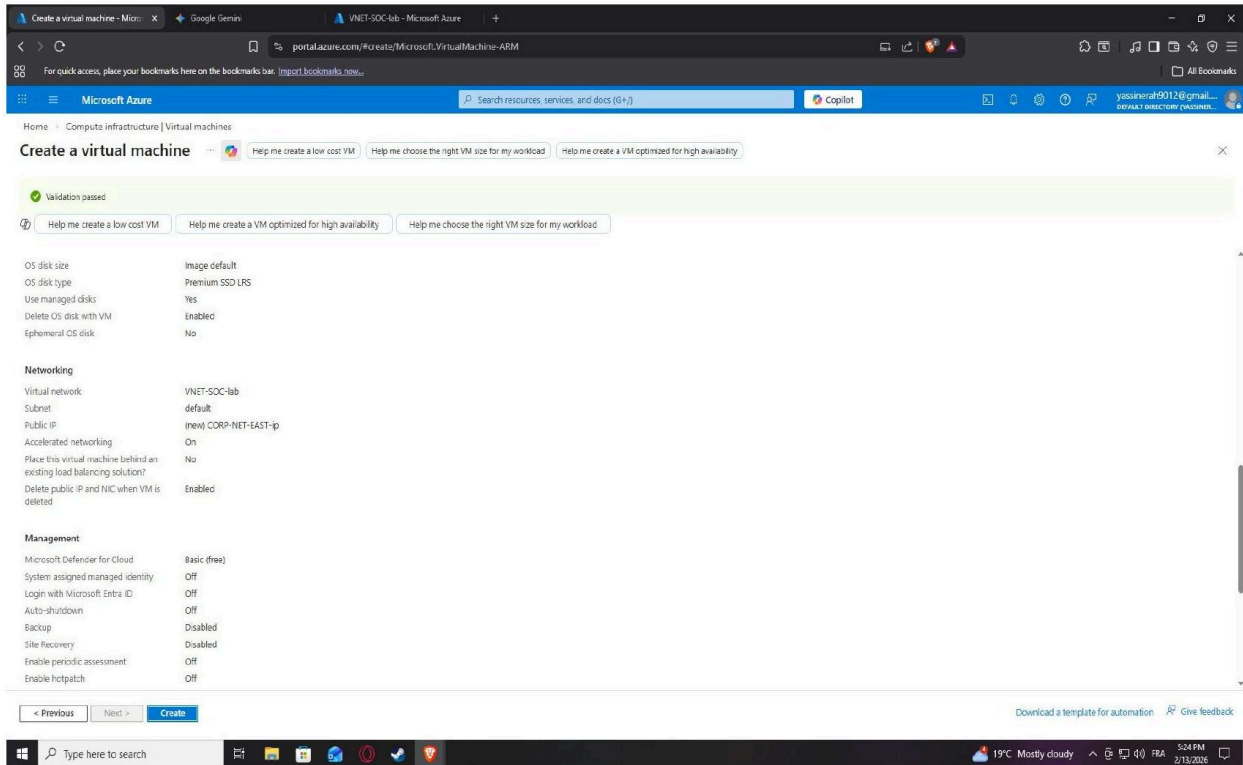


Figure 5: Complete Resource Group Inventory

Figure 4 reveals additional configuration details for storage and network connectivity:

- Storage configuration: Premium SSD LRS disk with image default sizing
- Managed disk utilization: Azure-managed disks enabled
- Network placement: Connected to VNET-SOC-lab/default subnet
- Public IP assignment: New public IP (CORP-NET-EAST-ip) created for remote access
- Accelerated networking: Enabled for improved network performance
- Load balancing: Not configured (standalone instance)
- Public IP retention: Enabled to preserve IP address when VM is deleted

Step 4: Network Security Group Configuration

Following virtual machine deployment, the laboratory infrastructure was expanded with additional network resources. Figure 5 displays the complete resource

inventory within the MYI-SOC-lab resource group, demonstrating the successful deployment of all dependent infrastructure components.

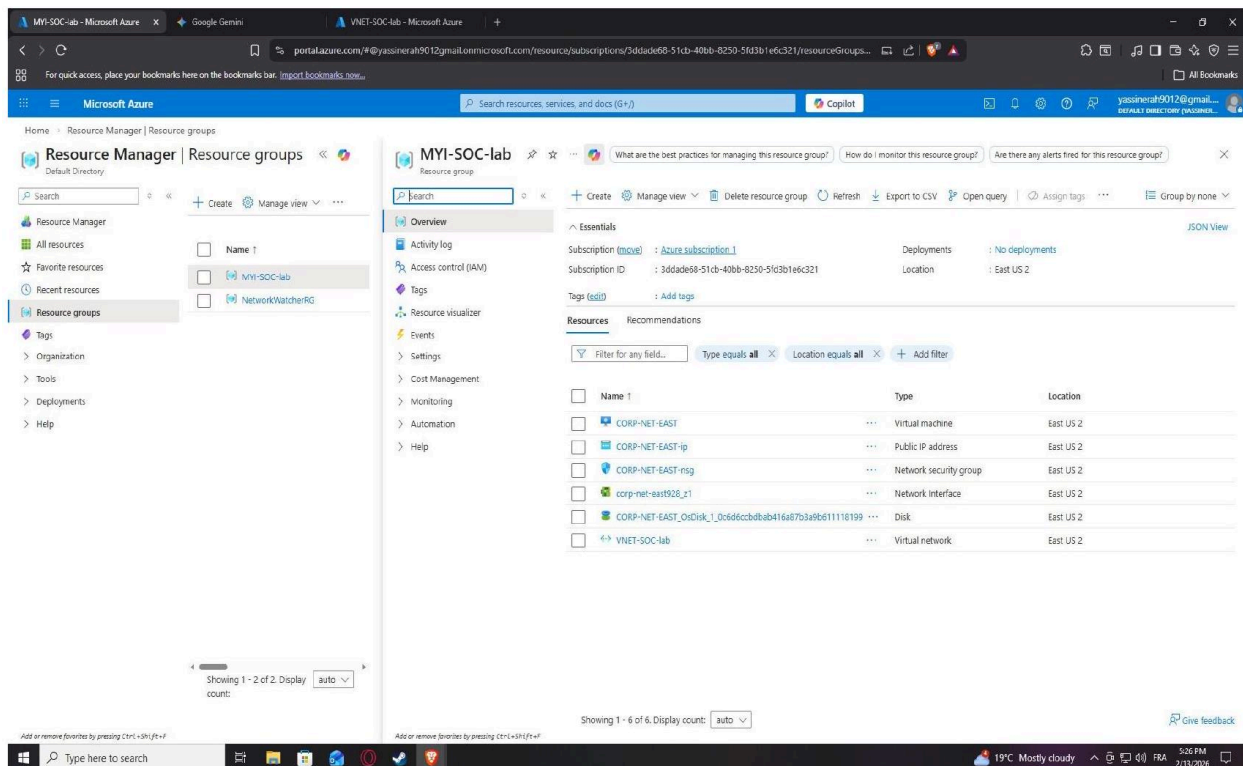


Figure 6: Network Security Group - Security Rule Configuration

The resource group now contains six distinct resources, all deployed in the East US 2 region:

1. CORP-NET-EAST - Virtual machine (primary monitored endpoint)
2. CORP-NET-EAST-ip - Public IP address for external connectivity
3. CORP-NET-EAST-nsg - Network security group for firewall rules
4. corp-net-east928_z1 - Network interface for VM connectivity
5. CORP-NET-EAST_OsDisk_1_0c6d6cddb46a87b3a9b6... - OS disk storage
6. VNET-SOC-lab - Virtual network infrastructure

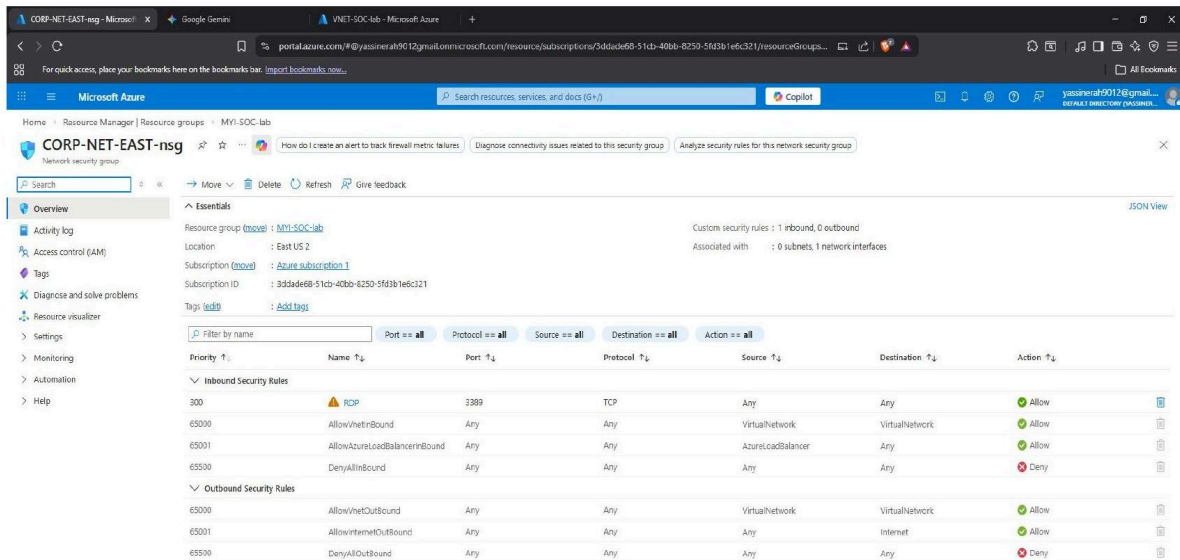


Figure 6 demonstrates the network security group configuration for the CORP-NET-EAST-nsg resource. This configuration implements a deliberately permissive security posture to facilitate penetration testing and security event generation. The security rules are organized into two categories:

Inbound Security Rules:

- Priority 300 - RDP access (Port 3389/TCP) from any source - WARNING: Security risk intentionally configured
- Priority 65000 - AllowVnetInBound (default rule permitting intra-VNet traffic)
- Priority 65001 - AllowAzureLoadBalancerInBound (Azure infrastructure)
- Priority 65500 - DenyAllInBound (default deny rule)

Outbound Security Rules:

- Priority 65000 - AllowVnetOutBound (permit outbound to virtual network)
- Priority 65001 - AllowInternetOutBound (permit outbound to Internet)
- Priority 65500 - DenyAllOutBound (default deny rule)

Step 5: Intentionally Vulnerable Network Configuration

To generate comprehensive security event telemetry and simulate real-world attack scenarios, a custom inbound security rule was implemented with highly

permissive access controls. This configuration represents a deliberate security misconfiguration designed to attract unauthorized access attempts and generate security events for SIEM analysis.

Figure 7: Custom Inbound Security Rule - DANGER Configuration

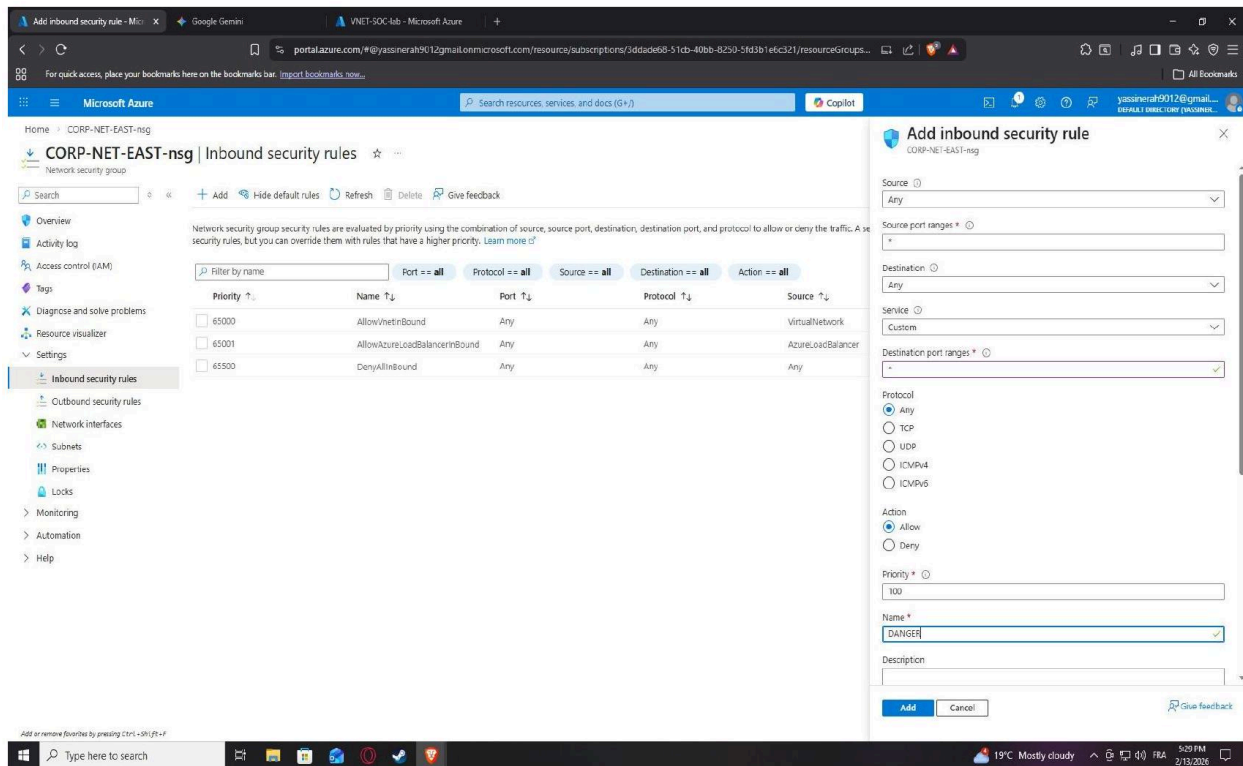


Figure 7 displays the custom security rule creation interface with the following highly permissive configuration:

- Source: Any (0.0.0.0/0) - Accepts traffic from any Internet source
- Source port ranges: * (any port)
- Destination: Any - No IP filtering applied
- Service: Custom (all protocols and ports enabled)
- Destination port ranges: * (all ports exposed)
- Protocol: Any (TCP, UDP, ICMP, and all other protocols permitted)
- Action: Allow - Permits all matching traffic

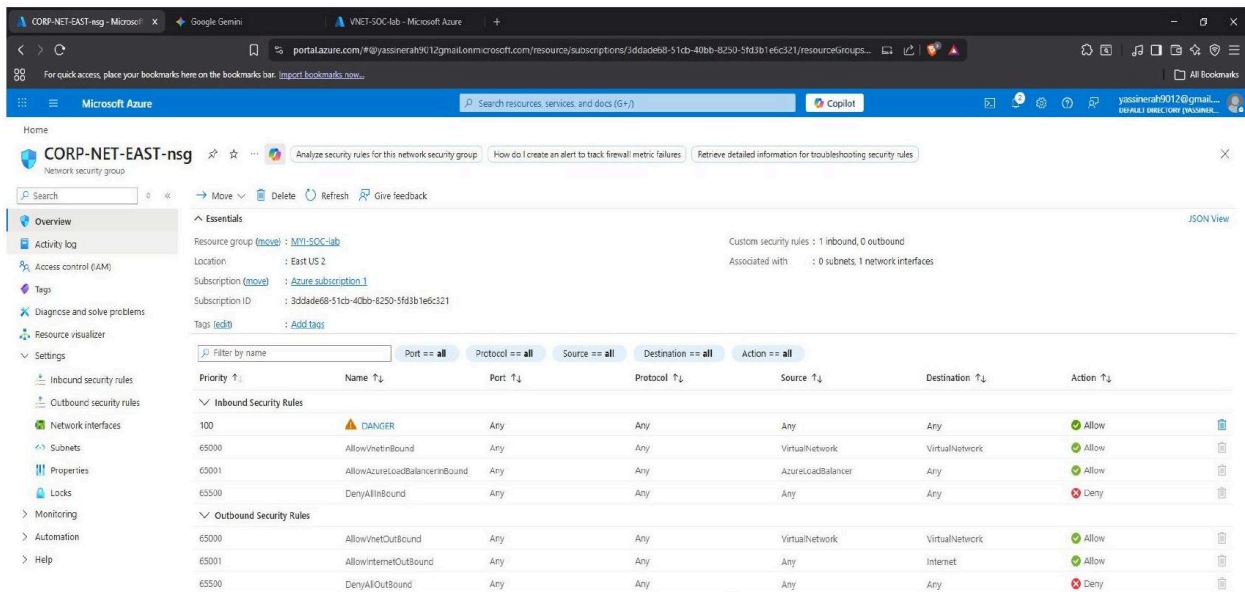
- Priority: 100 (highest priority, processes before RDP rule at 300)
- Name: DANGER - Explicitly named to indicate security risk

Security Impact Assessment:

This configuration completely disables network-level access controls and exposes all TCP, UDP, and other protocol services to unrestricted Internet access. In a production environment, this configuration would represent a critical security vulnerability (CVSS 10.0). For laboratory purposes, this enables:

- Automated attack detection and logging
- Brute force authentication attempt monitoring
- Port scanning activity analysis
- Malicious payload delivery detection
- Comprehensive security event generation for SIEM correlation

Figure 8: Updated Network Security Group Rules



Priority	Name	Port	Protocol	Source	Destination	Action
100	DANGER	Any	Any	Any	Any	Allow
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork	Allow
65001	AllowAzureLoadBalancerInBound	Any	Any	AzureLoadBalancer	Any	Allow
65500	DenyAllInBound	Any	Any	Any	Any	Deny
65000	AllowVnetOutBound	Any	Any	VirtualNetwork	VirtualNetwork	Allow
65001	AllowInternetOutBound	Any	Any	Any	Internet	Allow
65500	DenyAllOutBound	Any	Any	Any	Any	Deny

Figure 8 confirms the successful deployment of the DANGER security rule at priority 100 , positioned above the existing RDP rule at priority 300 . The orange warning indicator highlights the custom rule as requiring configuration attention, accurately reflecting its non-standard security posture.

Step 6: Virtual Machine Connectivity Validation

Following the network security configuration, the virtual machine's operational status and remote connectivity capabilities were validated through the Azure Portal and Remote Desktop Protocol (RDP) connection.

Figure 9: Virtual Machine Status - Running State

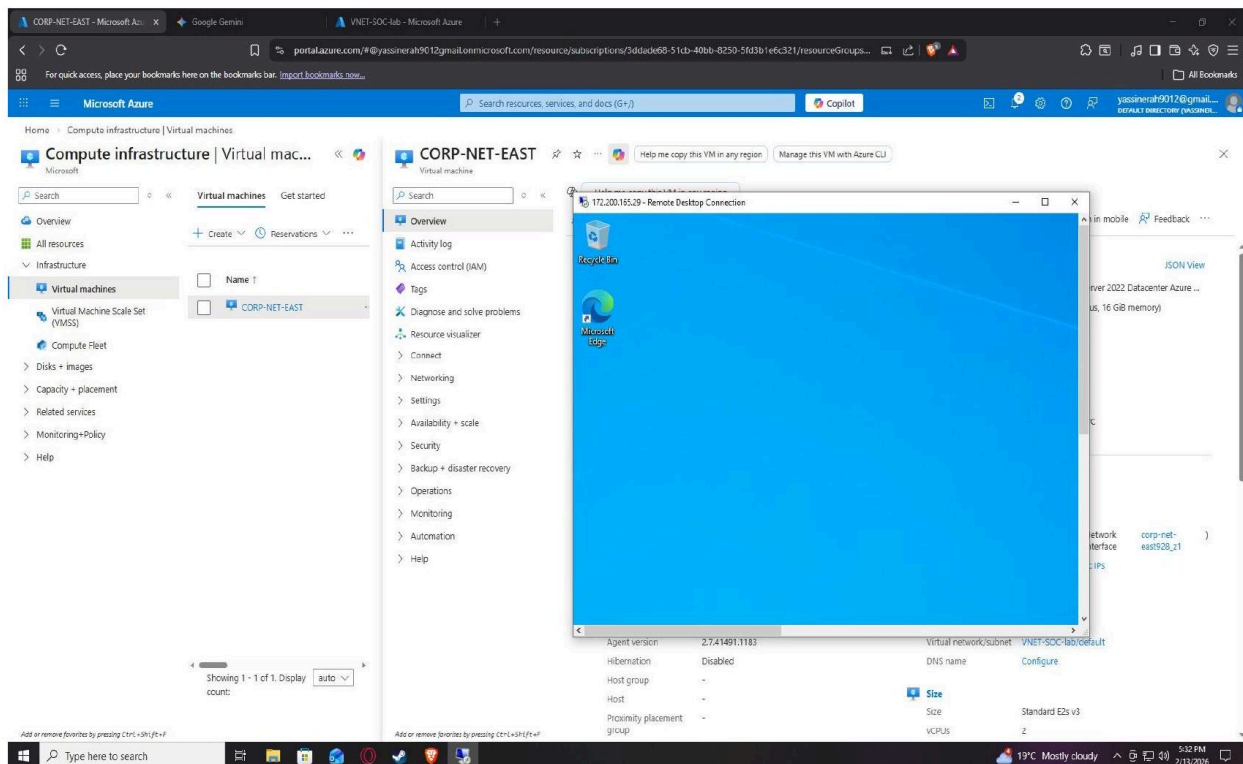


Figure 9 displays the virtual machine overview panel confirming successful deployment and operational status. Key operational parameters visible include:

- Status: Running (virtual machine successfully started)
- Operating system: Windows Server 2022 Datacenter Azure Edition - Gen2
- Compute size: Standard E2s v3 (2 vCPUs, 16 GiB memory)
- Public IP address: 172.200.165.29 (accessible for RDP connectivity)
- Private IP address: 10.0.0.4 (internal virtual network addressing)
- Virtual network/subnet: VNET-SOC-lab/default
- DNS name: Configure option available

Figure 10: Remote Desktop Protocol Connection

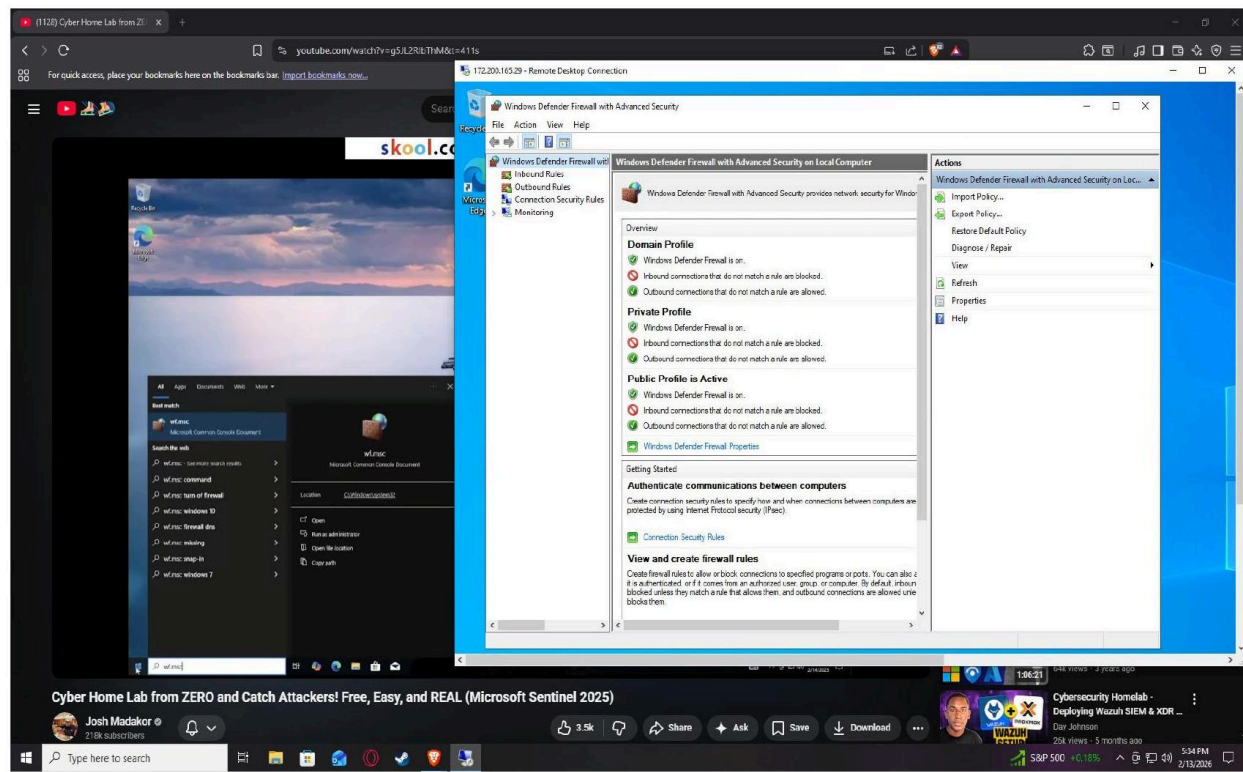


Figure 10 demonstrates successful Remote Desktop Protocol connectivity to the CORP-NET-EAST virtual machine. The screenshot shows the Windows Server 2022 desktop environment with an active Windows Defender Firewall with Advanced Security management console. This confirms:

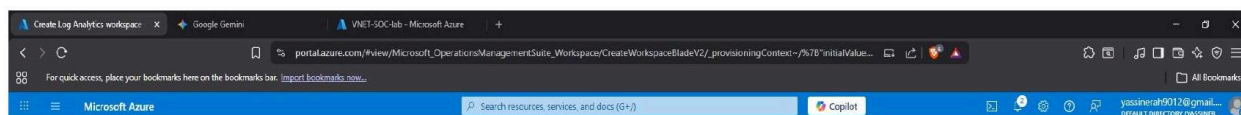
- Successful RDP authentication using configured credentials
- Network Security Group rules permitting inbound RDP traffic
- Operational Windows Server 2022 instance
- Windows Defender Firewall configured with Domain, Private, and Public profiles active

Step 7: Log Analytics Workspace Deployment

A dedicated Log Analytics workspace was provisioned to serve as the centralized repository for security event data collection, storage, and analysis. This

workspace provides the foundational data infrastructure required for Microsoft Sentinel SIEM capabilities.

Figure 11: Log Analytics Workspace Configuration



Home Log Analytics workspaces

Create Log Analytics workspace



Basics

Subscription	Azure subscription 1
Resource group	MY-SOC-lab
Name	LAW-SOC-lab
Region	East US 2

Pricing

Pricing tier Pay-as-you-go (Per GB 2018)

The cost of your workspace depends on the volume of data ingested and how long it is retained. Regional pricing details are available on the Azure Monitor pricing page. You can change to a different pricing tier after the workspace is created.

Learn more about Log Analytics pricing models.

Tags

None

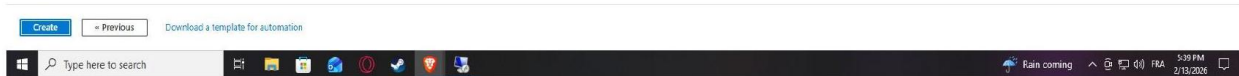


Figure 11 presents the final validation screen for Log Analytics workspace creation with the following configuration parameters:

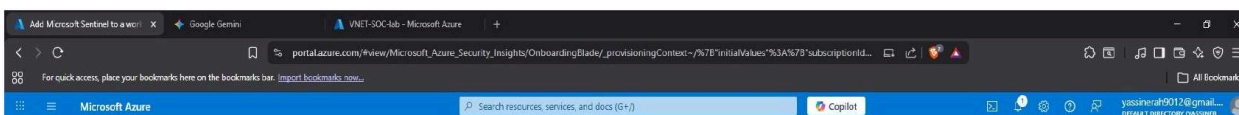
- Workspace name: LAW-SOC-lab
- Subscription: Azure subscription 1
- Resource group: MYI-SOC-lab (co-located with security infrastructure)
- Region: East US 2 (matching virtual machine placement)
- Pricing tier: Pay-as-you-go (Per GB 2018)

The Pay-as-you-go pricing model provides cost flexibility for laboratory environments with variable data ingestion rates. Pricing is determined by the volume of data ingested (measured in GB) rather than fixed monthly costs, enabling cost-effective security monitoring for development and testing scenarios.

Step 8: Microsoft Sentinel SIEM Deployment

Microsoft Sentinel was deployed and configured to provide cloud-native Security Information and Event Management (SIEM) and Security Orchestration, Automation, and Response (SOAR) capabilities. The deployment process integrates Sentinel with the previously created Log Analytics workspace to enable comprehensive security monitoring.

Figure 12: Microsoft Sentinel Workspace Selection



Home Microsoft Sentinel

Add Microsoft Sentinel to a workspace

- Create a new workspace ☐ Refresh

Microsoft Sentinel offers a 31-day free trial. See [Microsoft Sentinel pricing](#) for more details.

New Microsoft Sentinel workspaces created by authorized users are automatically onboarded and redirected to the Defender portal, [Learn more](#)

Workspace ↑↓	Location ↑ ↓	ResourceGroup ↑↓	Subscription ↑↓	Directory ↑↓
fiP LAW-SOC-lab	eastus2	myi-soc-lab	Azure subscription 1	Default Directory

Figure 12 displays the Microsoft Sentinel workspace association interface. The LAW-SOC-lab workspace is shown in the workspace selection table with the following attributes:

- Workspace: LAW-SOC-lab
- Location: eastus2 (East US 2 region)
- ResourceGroup: myi-soc-lab
- Subscription: Azure subscription 1
- Directory: Default Directory

A notification banner indicates that Microsoft Sentinel offers a 31-day free trial period. Additionally, the interface notes that new Microsoft Sentinel workspaces created by authorized users are automatically onboarded and redirected to the Defender portal for unified security management.

Figure 13: Microsoft Sentinel Getting Started Interface

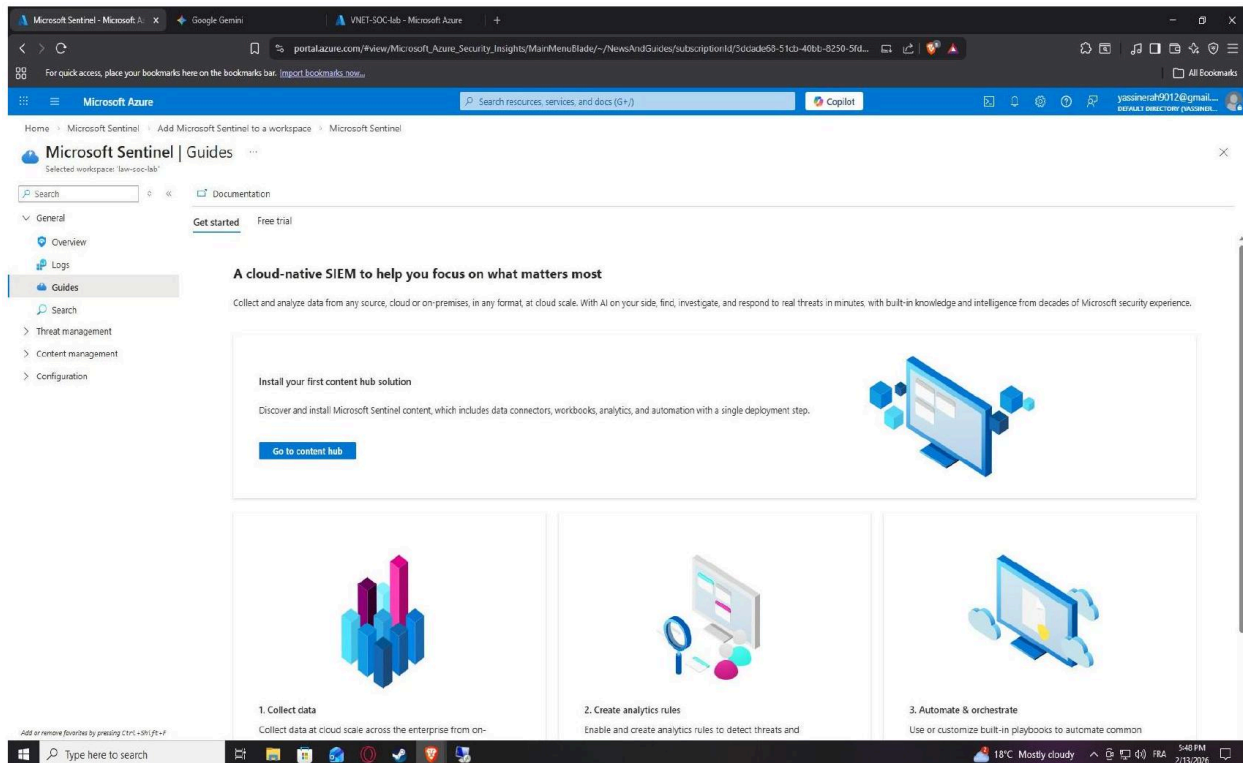


Figure 13 presents the Microsoft Sentinel initial configuration guide, describing Sentinel as ‘a cloud-native SIEM to help you focus on what matters most.’ The interface outlines three primary operational workflows:

1. Collect data - Aggregate telemetry from any source (cloud or on-premises) in any format at cloud scale
2. Create analytics rules - Enable and create analytic rules to detect threats and anomalies with AI assistance
3. Automate & orchestrate - Use or customize built-in playbooks to automate common procedures and responses

The interface provides a direct navigation option to ‘Go to content hub’ for deploying the first content hub solution, which includes data connectors, workbooks, analytics, and automation capabilities in a single deployment package.

Step 9: Windows Security Events Data Connector Configuration

The Windows Security Events data connector was configured to ingest security event logs from the CORP-NET-EAST virtual machine into the Log Analytics workspace. This connector enables Microsoft Sentinel to collect, normalize, and analyze Windows security events for threat detection and investigation.

Figure 14: Sentinel Content Hub - Security Event Solutions

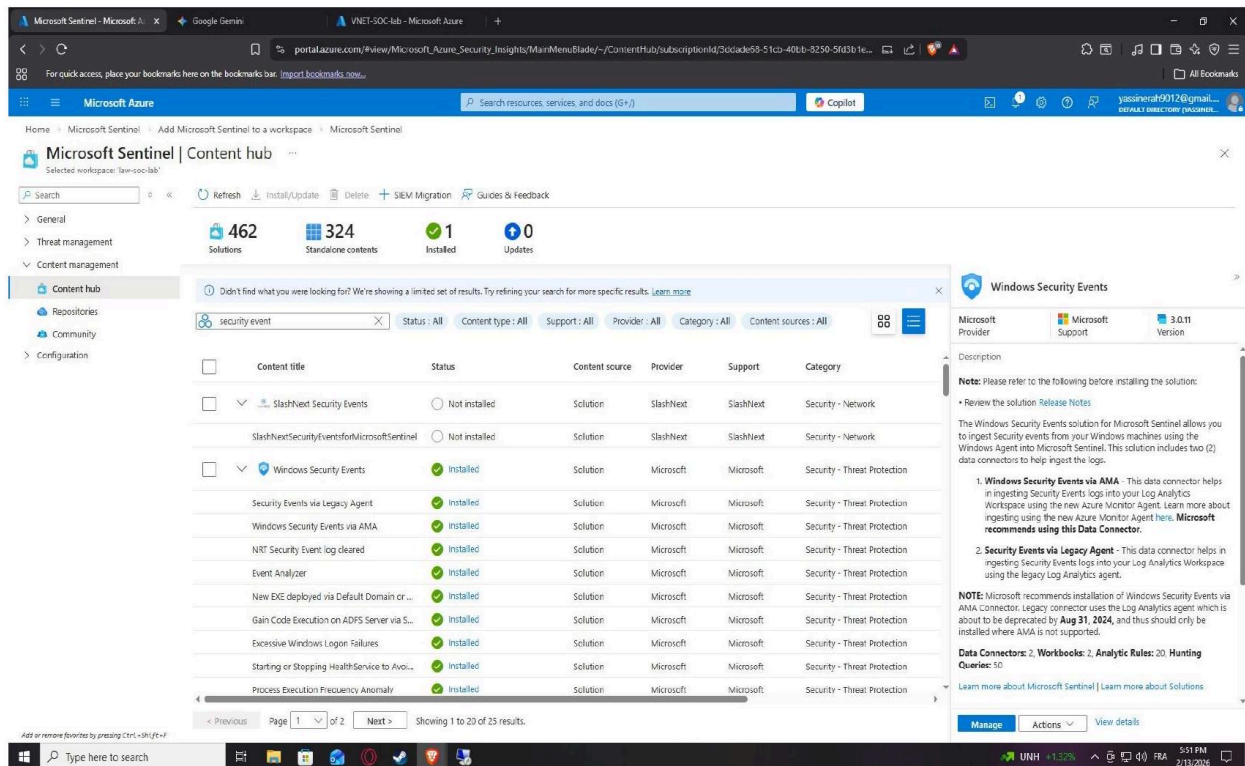


Figure 14 displays the Microsoft Sentinel Content Hub with the 'security event' search query applied. The search results present multiple available solutions:

- Windows Security Events (462 Solutions) - Primary data connector solution
- Legacy solutions for backward compatibility

The Windows Security Events solution includes:

- 2 Data Connectors (Windows Security Events via AMA and Legacy Agent)
- 2 Workbooks for security event visualization
- 2 Analytic Rules for automated threat detection
- 20 Hunting Queries for proactive threat hunting

- Version 3.0.11
- Provider: Microsoft Corporation

Figure 15: Windows Security Events Solution Details

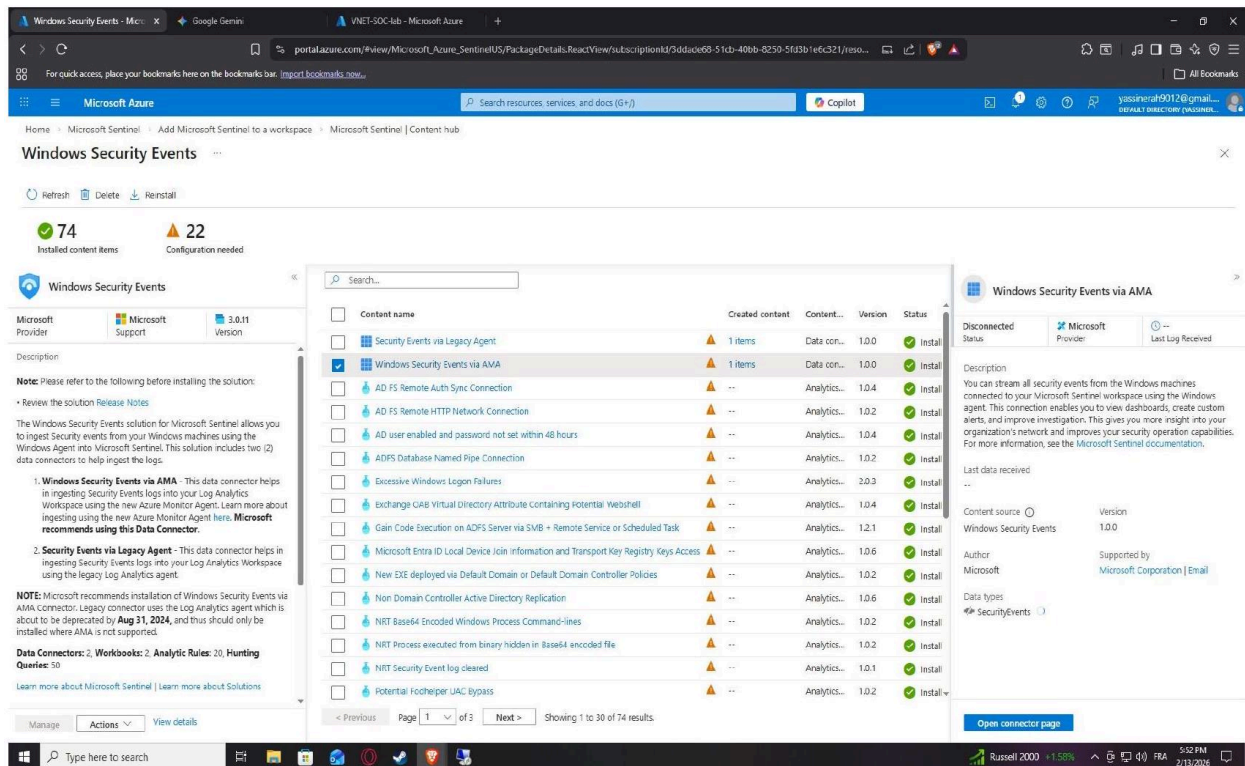


Figure 15 presents the detailed content inventory for the Windows Security Events solution. The solution contains 74 installed content items and 22 items requiring configuration:

Installed Data Connectors:

- Security Events via Legacy Agent (installed)
- Windows Security Events via AMA (installed)

Analytic Rules Templates:

Multiple pre-configured detection rules for common attack patterns, including:

- AD FS Remote Auth Sync Connection

- AD FS Remote HTTP Network Connection
- AD user enabled and password not set within 48 hours
- ADFS Database Named Pipe Connection
- Excessive Windows Logon Failures
- Gain Code Execution on ADFS Server via SMB + Remote Service or Scheduled Task
- Microsoft Entra ID Local Device Join Information and Transport Key Registry Keys Access
- New EXE deployed via Default Domain or Default Domain Controller Policies
- Non Domain Controller Active Directory Replication
- NRT Process executed from binary hidden in Base64 encoded file
- NRT Security Event log cleared
- Potential Fodhelper UAC Bypass
- Starting or Stopping HealthService to Avoid Detection

Step 10: Azure Monitor Agent (AMA) Connector Configuration

The Windows Security Events via AMA data connector was configured to establish the connection between the CORP-NET-EAST virtual machine and the Log Analytics workspace. The Azure Monitor Agent (AMA) represents the current generation agent architecture, replacing the legacy Log Analytics agent.

Figure 16: Windows Security Events via AMA - Configuration Prerequisites

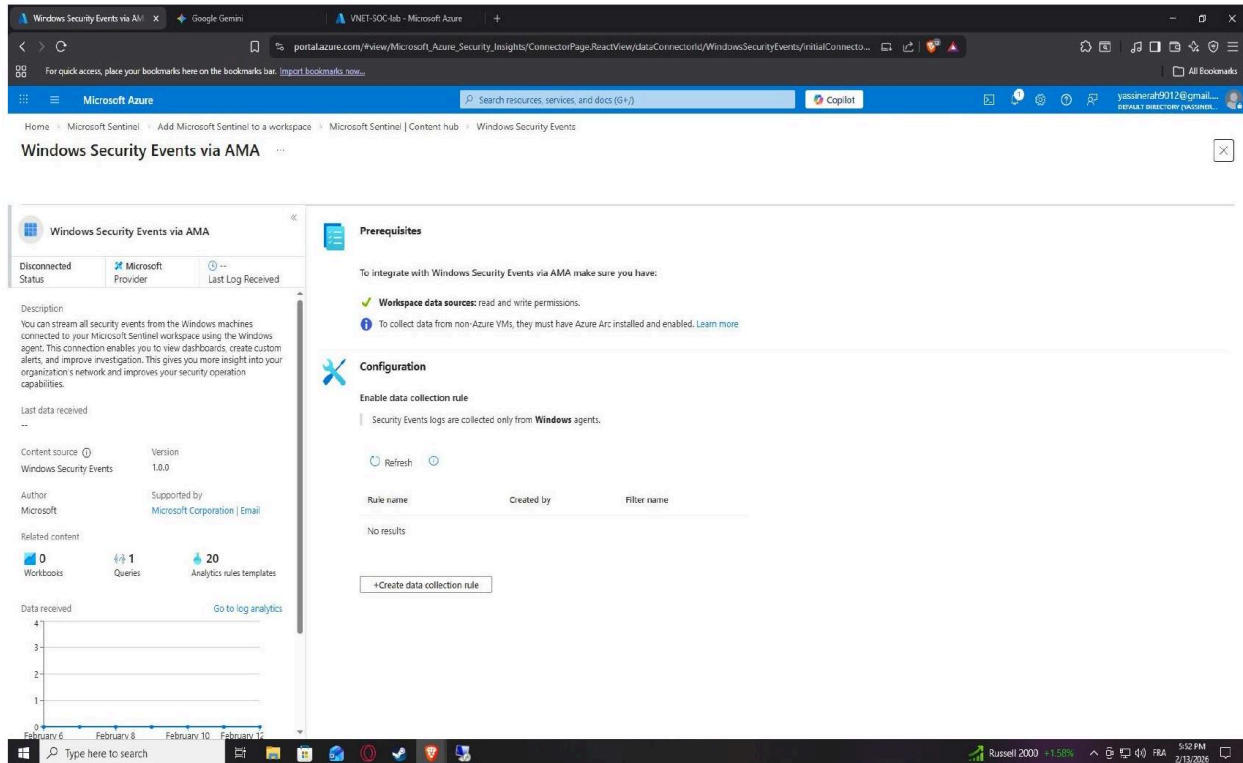


Figure 16 presents the data connector configuration interface with important prerequisites and configuration guidance:

Prerequisites:

- Workspace data sources: Read and write permissions required
- Non-Azure VM requirement: Azure Arc installation mandatory for data collection

Configuration Notes:

- Security Events logs are collected exclusively from Windows agents
- Microsoft recommends installing Windows Security Events via AMA Connector
- Legacy connector uses the deprecated Log Analytics agent (retirement: August 31, 2024)
- The recommendation banner links to Microsoft Sentinel documentation for additional guidance

The interface displays two data connector options:

1. Security Events via Legacy Agent - Legacy connector using deprecated agent
2. Windows Security Events via AMA - Current generation connector (selected for configuration)

Step 11: Data Collection Rule (DCR) Creation

A Data Collection Rule (DCR) was created to define which security events should be collected from Windows machines and forwarded to the Log Analytics workspace. DCRs provide granular control over event filtering, reducing storage costs and noise while ensuring relevant security events are captured.

Figure 17: Data Collection Rule Configuration Summary

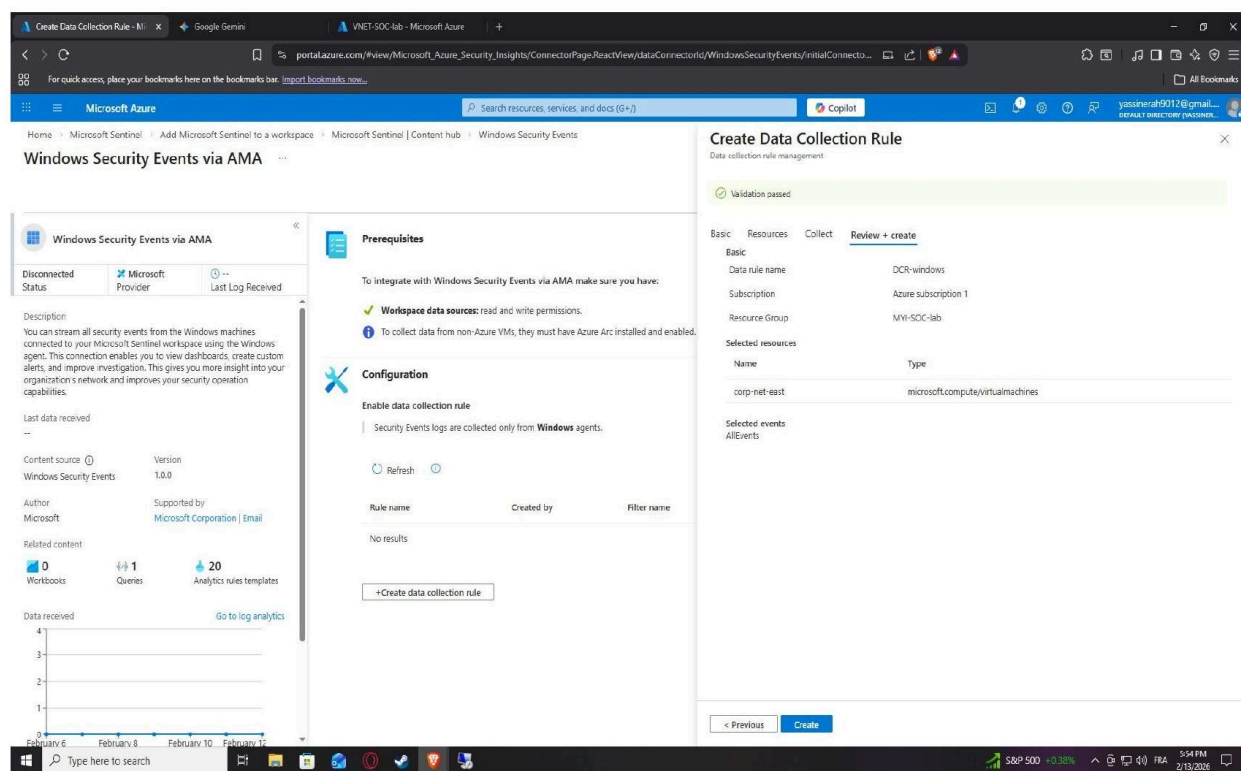


Figure 17 displays the final validation screen for Data Collection Rule creation with the following configuration:

Basic Configuration:

- Data rule name: DCR-windows
- Subscription: Azure subscription 1
- Resource Group: MYI-SOC-lab

Selected Resources:

- Name: corp-net-east
- Type: microsoft.compute/virtualmachines

Selected Events:

- AllEvents - Comprehensive event collection (includes all Windows Security Events)

The 'AllEvents' selection configures the data collection rule to capture all Windows Security Event log entries, providing maximum visibility into system activity. While this configuration generates higher data volumes compared to filtered event sets, it ensures comprehensive security monitoring capabilities for threat detection and forensic analysis.

Step 12: Log Analytics Query Verification

Following the data collection rule deployment, the Log Analytics workspace was queried using Kusto Query Language (KQL) to verify successful security event ingestion and analyze initial data collection results.

Figure 18: Log Analytics Workspace - SecurityEvent Query Results

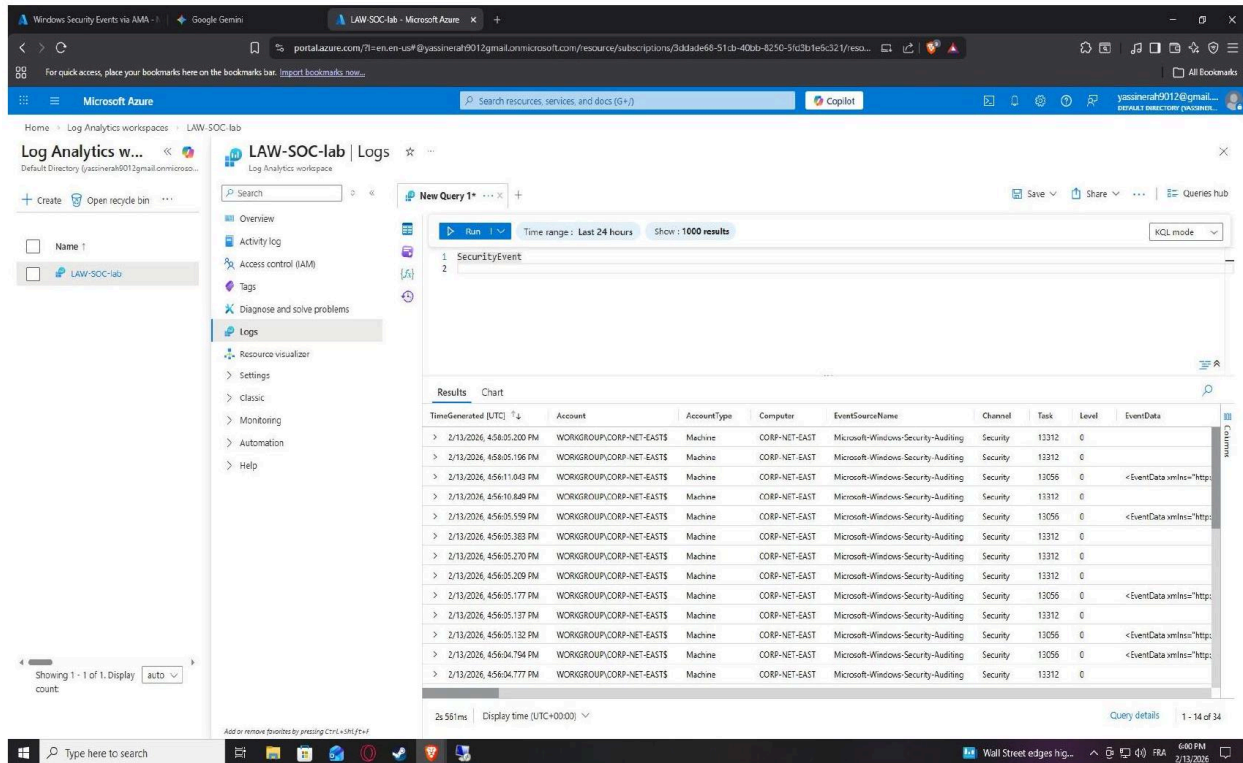


Figure 18 displays the Log Analytics Logs interface with the basic SecurityEvent table query. The query interface shows:

Query Parameters:

- Time range: Last 24 hours
- Query: SecurityEvent
- Results: 1000 results returned (maximum display limit)

Sample Event Data Fields:

- TimeGenerated [UTC] - Event timestamp
- Account - Security principal (WORKGROUP\CORP-NET-EAST\$)
- AccountType - Machine account classification
- Computer - Source system (CORP-NET-EAST)
- EventSourceName - Microsoft-Windows-Security-Auditing

- Channel - Security event log channel
- Task - Event category identifier (13312, 13056)
- Level - Event severity (0 = Information)
- EventData - XML-formatted event details

The successful query execution with 34 total records confirms that:

- Azure Monitor Agent is properly installed and configured
- Data Collection Rule is actively forwarding security events
- Log Analytics workspace is receiving and indexing event data
- Security event telemetry is available for SIEM analysis

Step 13: Advanced KQL Query Analysis

Additional Kusto Query Language (KQL) queries were executed to extract specific security-relevant information from the collected event data, demonstrating analytical capabilities for threat detection and security monitoring.

Figure 19: Filtered Security Event Query with Projection

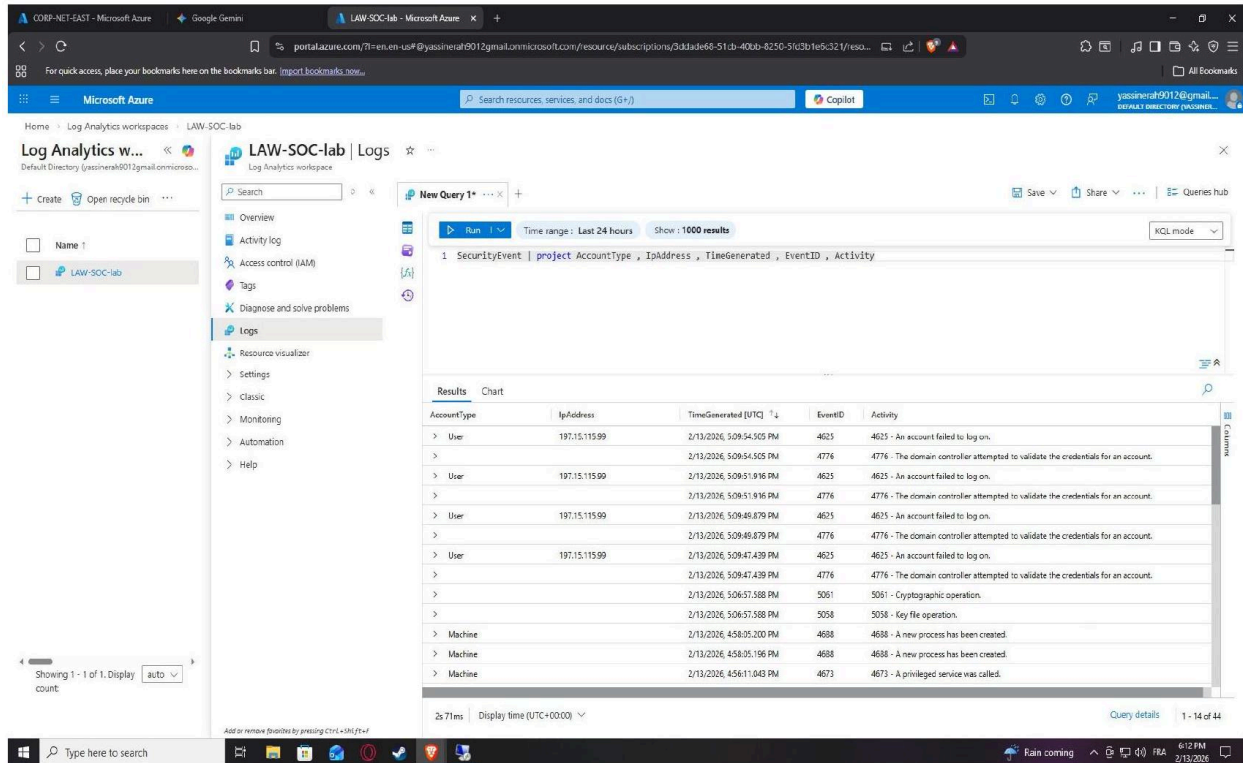


Figure 19 demonstrates a more sophisticated KQL query with field projection and result ordering:

KQL Query:

```
SecurityEvent | project AccountType, IpAddress, TimeGenerated, EventID, Activity
```

Query Explanation:

- project operator selects specific columns from the SecurityEvent table
- Reduces result set to relevant fields: AccountType, IpAddress, TimeGenerated, EventID, Activity
- Improves query performance by limiting data transfer
- Enhances readability for security analysis

Sample Results:

The query returned 44 records showing various account types (User, Machine), IP addresses, timestamps, event IDs (4625 - failed logon, 4776 - credential validation failure, 4778 - session reconnection, 5061 - cryptographic operation, 5058 - key file operation, 4673 - privileged service called, 4688 - process creation, 4672 - special privileges assigned), and activity descriptions.

Figure 20: User Account Failed Logon Analysis

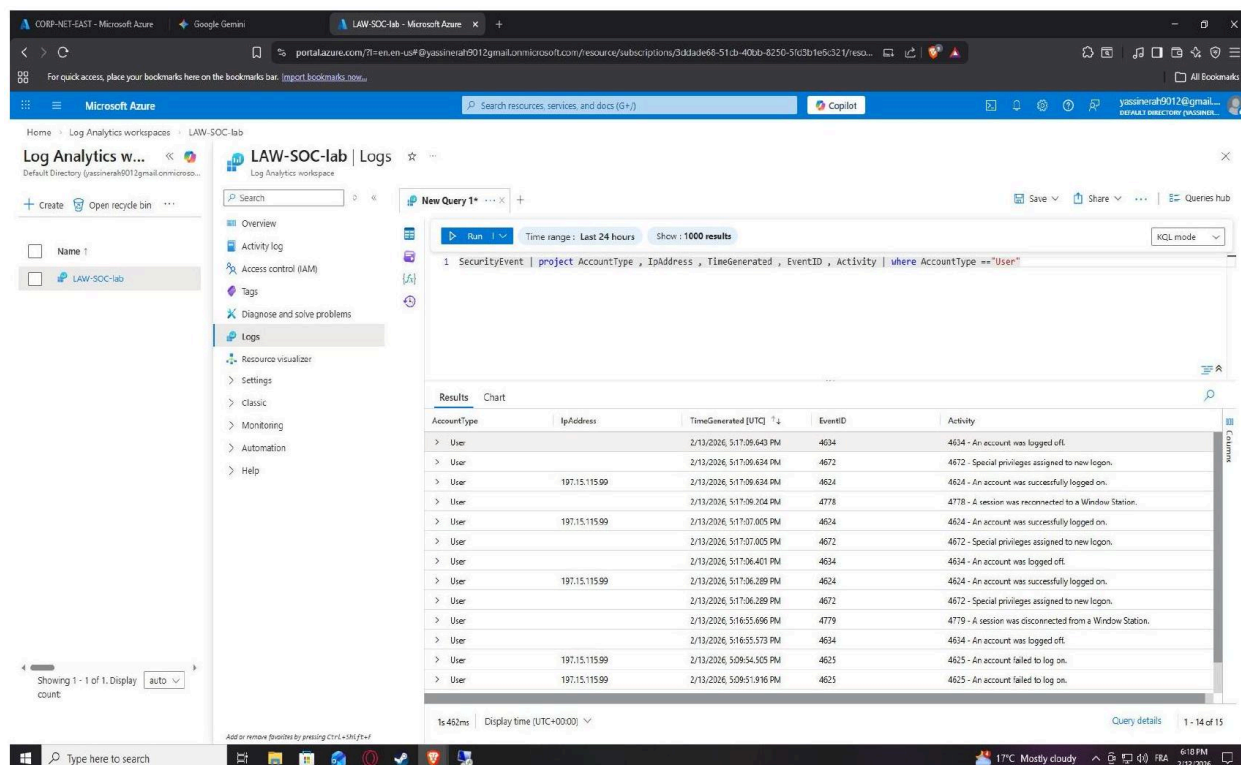


Figure 20 presents a threat-focused query specifically targeting failed authentication attempts:

KQL Query:

```
SecurityEvent | project AccountType, IpAddress, TimeGenerated, EventID, Activity | where AccountType == "User"
```

Query Capabilities:

- Filters events to User account types only (excludes Machine and Service accounts)
- Focuses analysis on interactive user authentication events
- Enables identification of brute force attack patterns
- Tracks failed logon attempts with source IP correlation

Security Analysis Findings:

The query results show 15 user-related events over the 24-hour period, including:

- Event ID 4625 - Failed logon attempts from IP 197.15.115.99
- Event ID 4776 - Domain controller credential validation failures
- Event ID 4624 - Successful logon (reconnaissance validation)
- Event ID 4778 - Session reconnection to Windows Station
- Event ID 4634 - Account logoff
- Event ID 4672 - Special privileges assigned to new logon

Threat Indicator: The presence of Event ID 4625 (failed logon) originating from external IP address 197.15.115.99 indicates potential unauthorized access attempts. This pattern is consistent with credential stuffing or brute force attack activity targeting the RDP service exposed through the permissive Network Security Group configuration.

Part 2 — Threat Detection & Automated Response

Overview

Part 2 builds on top of the deployed SIEM infrastructure by implementing a full **detect-to-block** automated response pipeline. The goal is to automatically detect brute-force RDP attacks hitting the exposed Azure VM and block the attacking IPs in the Network Security Group — with zero human intervention.

The pipeline has three layers:

- **Visualisation** — a GeoIP-enriched world map workbook showing live attack origins

- **Detection** — a custom Sentinel analytics rule that fires when any IP generates 10+ failed logons in 5 minutes
- **Response** — a Logic App playbook that reads the incident's attacker IPs and injects deny rules directly into the NSG via the Azure REST API

End-to-end flow:

```

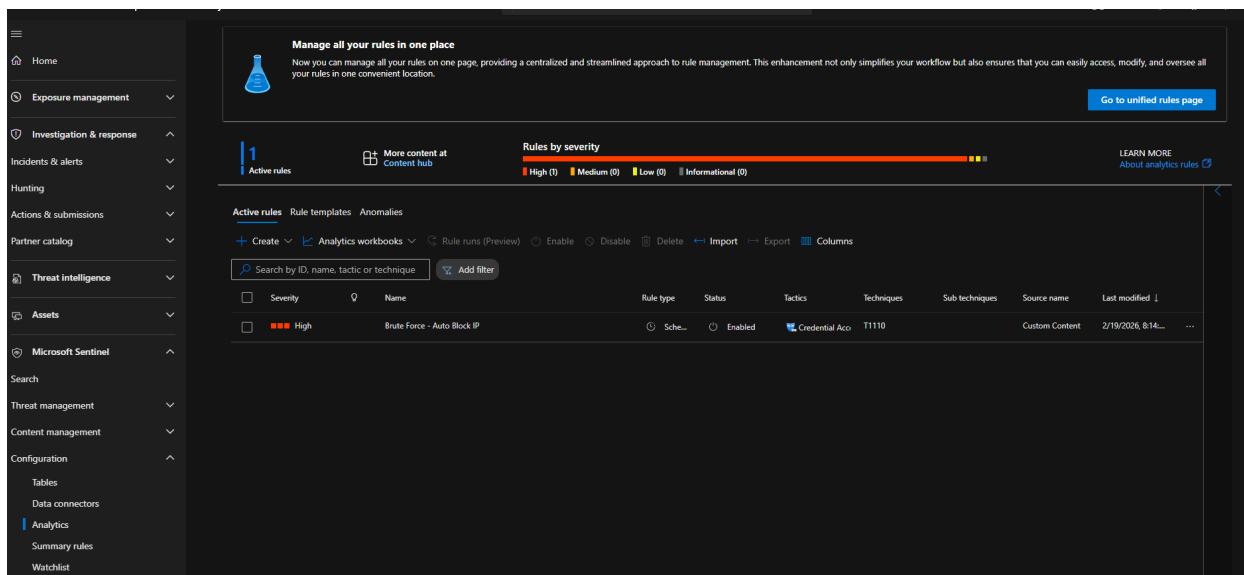
Windows VM (exposed to internet – DANGER NSG rule)
└─ Failed RDP logons → Event ID 4625 → Log Analytics
    └─ Analytics Rule fires (≥10 failures / 5 min)
        └─ Sentinel Incident created (IP entity extracted)
            └─ Automation rule triggers Logic App
                ├── GET current NSG rules
                ├── Find highest priority
                └─ PUT Deny rule per attacker IP → NSG
  
```

Detection Pipeline

Component	Name	Purpose
Watchlist	geoip	GeoIP lookup table for attack map enrichment
Workbook	Windows VM attack map	Live world map of attack origins
Analytics Rule	Brute Force - Auto Block IP	KQL-based detection, fires on ≥10 failed logons / 5 min
Logic App	LOG-SOC-lab	Automated playbook — blocks attacker IPs in NSG

Screenshot 1 — Analytics Rules Dashboard

Analytics Rules Dashboard



What this screenshot shows:

The Microsoft Sentinel Analytics rules dashboard inside the Microsoft Defender unified portal. It lists all active detection rules configured in the SIEM.

Its role in the lab:

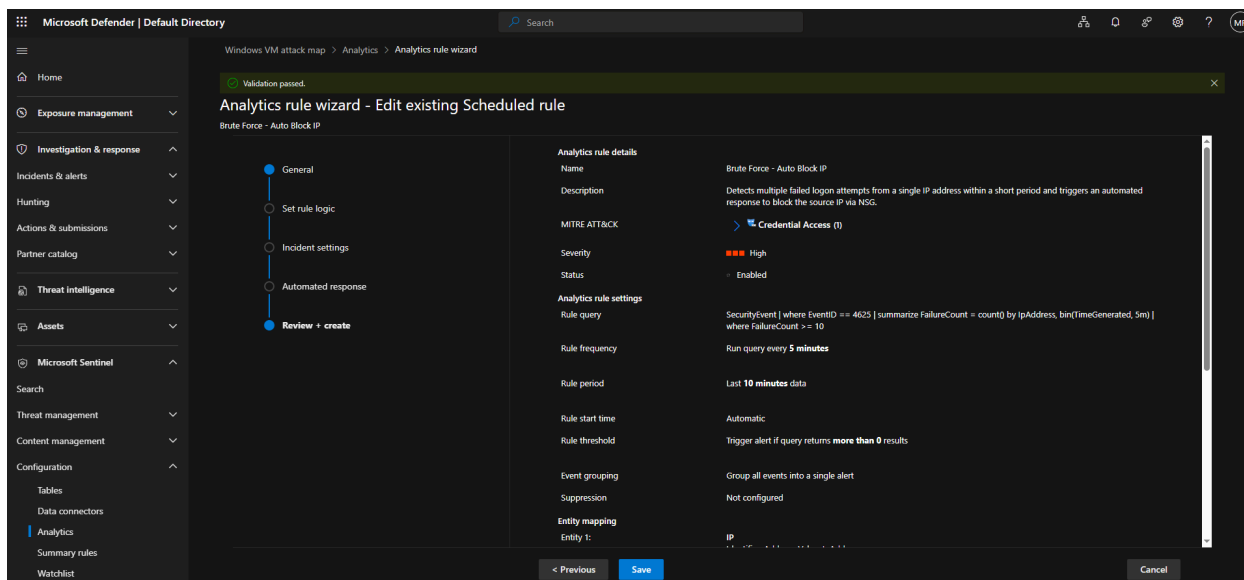
This confirms the custom rule **“Brute Force - Auto Block IP”** is live and enabled. It is the entry point of the entire automated response chain — without this rule firing, no incidents are created and the Logic App never executes.

Key details visible:

Field	Value
Severity	High
Rule type	Scheduled
Status	Enabled
Tactic	Credential Access — T1110
Source	Custom Content

Screenshot 2 — Analytics Rule Configuration (General & Query)

Analytics Rule General & Query



What this screenshot shows:

The Analytics Rule Wizard showing the full configuration of the **“Brute Force - Auto Block IP”** rule — its general settings, KQL detection query, and scheduling parameters.

Its role in the lab:

This defines the detection logic. The KQL query is the brain of the rule — it determines exactly what constitutes a brute-force event and when an alert should fire.

KQL detection query:

```
SecurityEvent
| where EventID == 4625
| summarize FailureCount = count() by IPAddress, bin(TimeGenerated, 5m)
| where FailureCount >= 10
```

This checks every 5 minutes whether any single IP has produced 10 or more failed logon attempts (Event ID 4625) in a 5-minute window. If true, an alert fires.

Scheduling configuration:

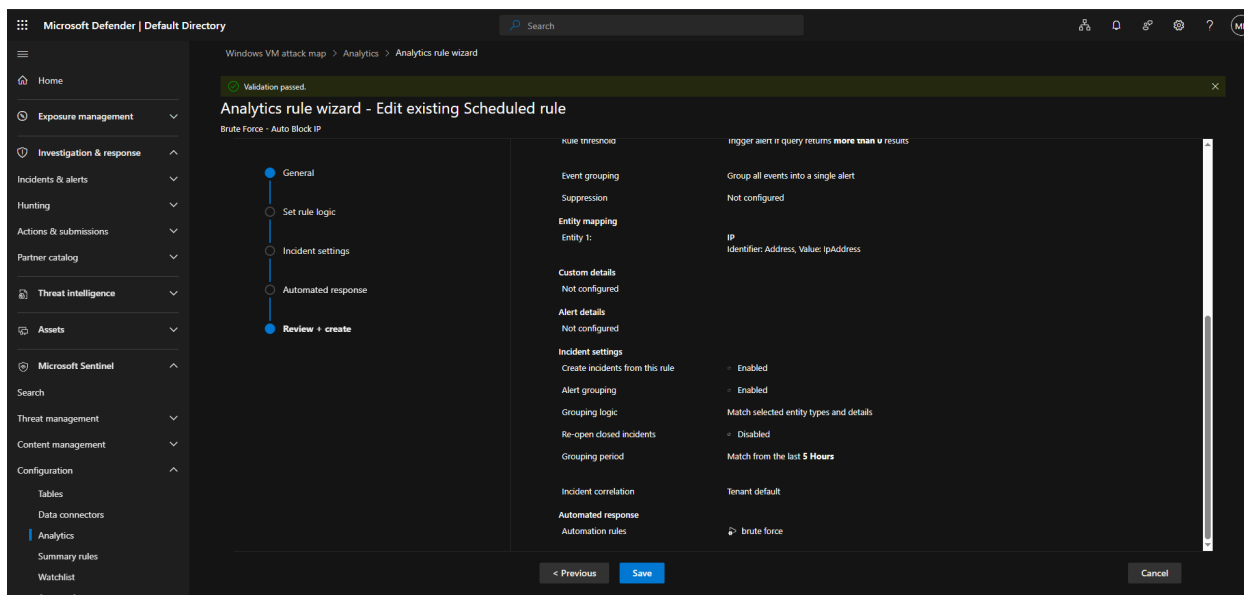
Parameter	Value
Run frequency	Every 5 minutes
Lookback period	Last 10 minutes

Parameter	Value
Alert threshold	> 0 results
Event grouping	All events into a single alert
Rule start time	Automatic

Event grouping is set to **all events into a single alert** to prevent alert flooding when a sustained brute-force attack is underway.

Screenshot 3 — Analytics Rule Configuration (Incident Settings & Automation)

Analytics Rule Incident Settings & Automation



What this screenshot shows:

The lower section of the same Analytics Rule Wizard — Entity Mapping, Incident Settings, and the Automated Response configuration.

Its role in the lab:

This section is the bridge between detection and automated response. Two things make the automation work:

1. Entity mapping:

- Entity: **IP** → Identifier: **Address** , Value: **IPAddress**

- This tells Sentinel to extract the attacker's IP address as a structured entity from every alert. Without this, the Logic App has no IP to block.

2. Incident settings:

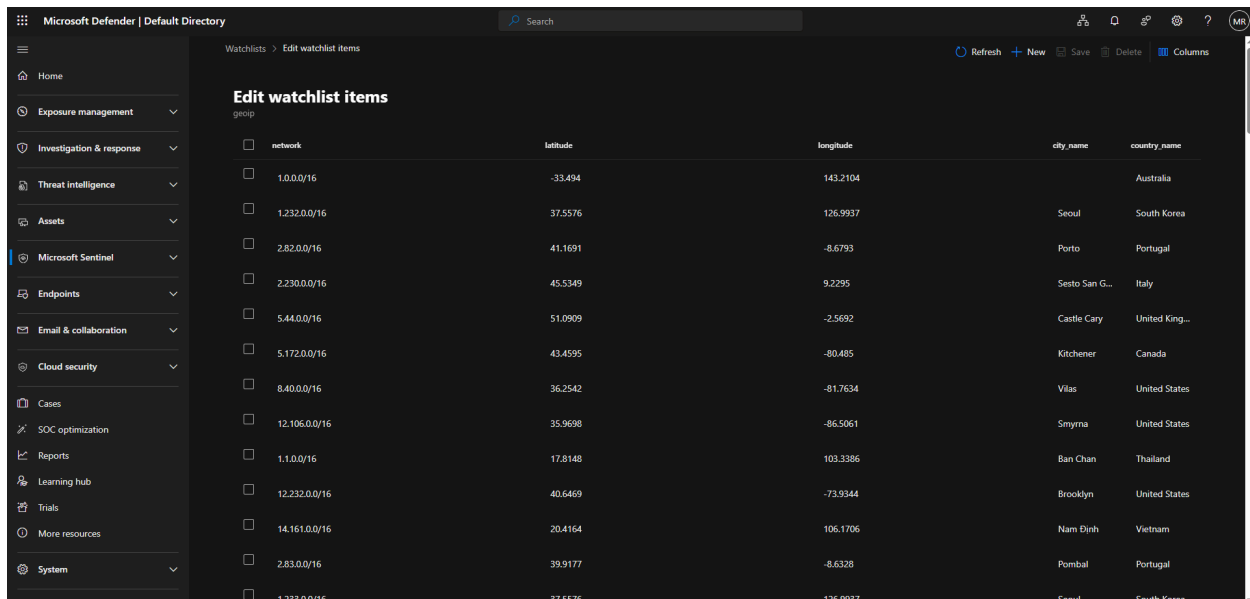
Parameter	Value
Create incidents	Enabled
Alert grouping	Enabled
Grouping logic	Match selected entity types and details
Grouping period	Last 5 hours
Re-open closed incidents	Disabled

3. Automated response:

- Automation rule: **brute force** — this automation rule triggers the Logic App playbook every time a new incident is created by this analytics rule. It is the final link connecting detection to enforcement.

Screenshot 4 — GeoIP Watchlist

GeoIP Watchlist



network	latitude	longitude	city_name	country_name
1.0.0.0/16	-33.494	143.2104		Australia
1.232.0.0/16	37.5576	126.9937	Seoul	South Korea
2.82.0.0/16	41.1691	-8.6793	Porto	Portugal
2.230.0.0/16	45.5349	9.2295	Sesto San G.	Italy
5.44.0.0/16	51.0909	-2.5692	Castle Cary	United King...
5.172.0.0/16	43.4595	-80.485	Kitchener	Canada
8.40.0.0/16	36.2542	-81.7634	Vilas	United States
12.106.0.0/16	35.9698	-86.5061	Smyrna	United States
1.1.0.0/16	17.8148	103.3386	Ban Chan	Thailand
12.232.0.0/16	40.6469	-73.9344	Brooklyn	United States
14.161.0.0/16	20.4164	106.1706	Nam Dinh	Vietnam
2.83.0.0/16	39.9177	-8.6328	Pombal	Portugal
1.233.0.0/16	37.5576	126.9937	Seoul	South Korea

What this screenshot shows:

The Sentinel Watchlist editor displaying the contents of the **geoip** watchlist — a

lookup table mapping IP network CIDR ranges to geographic locations. I got this list from an github repo.

Its role in the lab:

The GeoIP watchlist enables geographic enrichment of attacker IPs in the attack map workbook. Without it, Sentinel only knows an IP address. With it, Sentinel can translate that IP into a city, country, latitude, and longitude for map visualisation.

Schema:

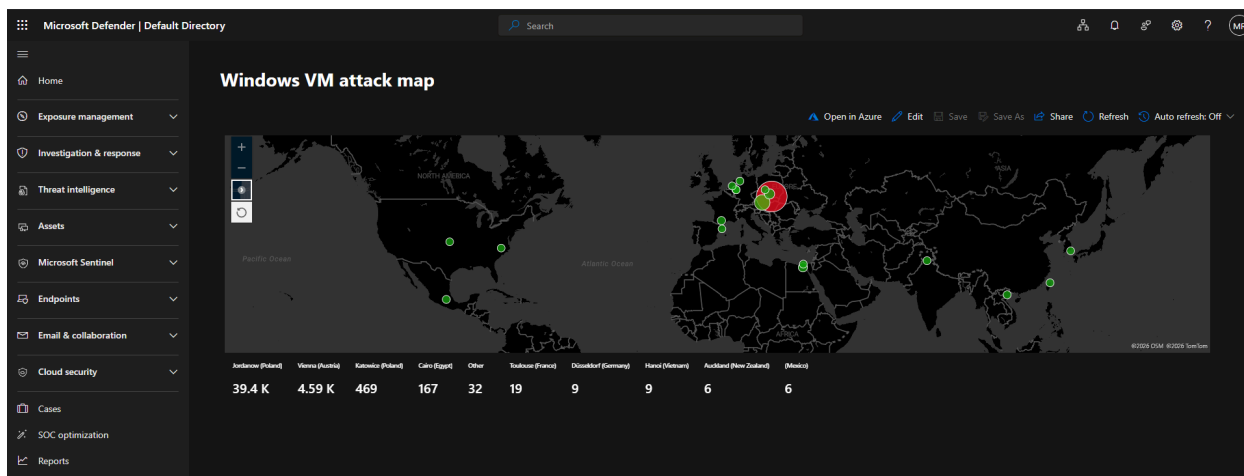
Field	Example	Purpose
<code>network</code>	<code>1.0.0.0/16</code>	CIDR range matched against attacker IPs
<code>latitude</code>	<code>-33.494</code>	Geographic coordinate for map plotting
<code>longitude</code>	<code>143.2104</code>	Geographic coordinate for map plotting
<code>city_name</code>	<code>Seoul</code>	Human-readable city label
<code>country_name</code>	<code>South Korea</code>	Country of origin

This watchlist is loaded in KQL via `_GetWatchlist("geoip")` and joined to the SecurityEvent table using `evaluate ipv4_lookup()`, which matches each attacker IP against the CIDR ranges to return its geographic location.

Sample origins visible in the watchlist: Australia, South Korea, Portugal, Italy, United Kingdom, Canada, United States, Thailand, Vietnam — reflecting the global nature of automated scanning activity.

Screenshot 5 — Attack Map Workbook

Attack Map Workbook



What this screenshot shows:

The custom Sentinel workbook **“Windows VM attack map”** — a live world map visualising the geographic origins of all failed RDP logon attempts against the lab VM, rendered as a green-to-red heatmap.

Its role in the lab:

This workbook is the primary threat intelligence visualisation tool. A SOC analyst opening this workbook immediately sees where attacks are coming from, at what volume, and which regions are most active — without running a single query manually.

How the map works:

- Each bubble represents a source location of failed logon attempts (Event ID 4625)
- Bubble **size** and **colour** both scale with **FailureCount** — larger and redder means more attacks
- The heatmap palette runs green (low) → red (high)
- The legend at the bottom ranks the top attack origins by failure count

Top attack origins observed:

Location	Failure Count
Jordanow, Poland	39,400
Vienna, Austria	4,590
Katowice, Poland	469
Cairo, Egypt	167

Location	Failure Count
Other (combined)	32+

The 39,400+ hits from a single Polish location within days of the VM being exposed demonstrates how quickly automated RDP brute-force botnets discover and target internet-facing Windows endpoints.

Screenshot 6 — Attack Map Workbook JSON

Attack Map JSON

```

1 {
2   "type": 3,
3   "name": "query - 0",
4   "content": {
5     "version": "KqlItem/1.0",
6     "query": "let GeoIPDB_FULL = _GetWatchlist('geoip');\nlet WindowsEvents = SecurityEvent;\nWindowsEvents | where EventID == 4625\n| order by TimeGenerated desc\n| evaluate ipv4_location",
7     "size": 3,
8     "timeContext": {
9       "durationMs": 259200000
10    },
11    "queryType": 0,
12    "resourceType": "microsoft.operationalinsights/workspaces",
13    "visualization": "map",
14    "mapSettings": {
15      "locInfo": "LatLong",
16      "locInfoColumn": "countryname",
17      "latitude": "latitude",
18      "longitude": "longitude",
19      "sizeSettings": "FailureCount",
20      "sizeAggregation": "Sum",
21      "opacity": 0.8,
22      "labelSettings": "friendly_location",
23      "legendMatrix": "FailureCount",
24      "legendAggregation": "Sum",
25      "itemColorSettings": {
26        "nodeColorField": "FailureCount",
27        "colorAggregation": "Sum",
28        "type": "heatmap",
29        "heatmapPalette": "greenRed"
30      }
31    }
32  }
33 }

```

What this screenshot shows:

The Advanced Editor view of the attack map workbook — the raw JSON configuration that defines the map visualization settings, including how data is positioned, sized, coloured, and labelled on the map.

Its role in the lab:

This JSON transforms raw KQL query results (a table of IPs, coordinates, and counts) into a geographic heatmap. Without this configuration, the query output is just a flat table with no visual representation.

Key JSON properties:

```

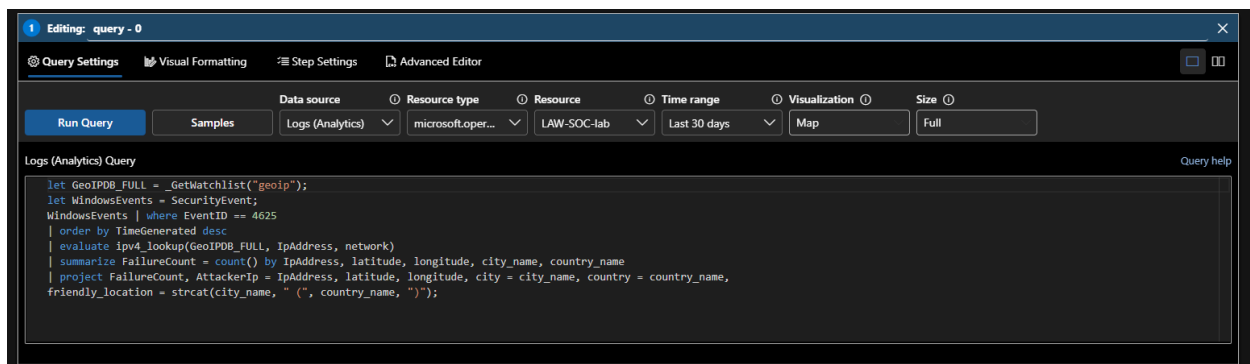
{
  "visualization": "map",
  "mapSettings": {
    "locInfo": "LatLong",
    "latitude": "latitude",
    "longitude": "longitude",
    "sizeSettings": "FailureCount",
    "sizeAggregation": "Sum",
    "opacity": 0.8,
    "labelSettings": "friendly_location",
    "legendMetric": "FailureCount",
    "legendAggregation": "Sum",
    "itemColorSettings": {
      "nodeColorField": "FailureCount",
      "colorAggregation": "Sum",
      "type": "heatmap",
      "heatmapPalette": "greenRed"
    }
  }
}

```

Property	Purpose
<code>locInfo: "LatLong"</code>	Uses latitude/longitude columns to position bubbles
<code>sizeSettings: "FailureCount"</code>	Bubble size scales with attack volume
<code>heatmapPalette: "greenRed"</code>	Green = low attack volume, Red = high
<code>opacity: 0.8</code>	Slight transparency so overlapping bubbles remain visible
<code>timeContext.durationMs: 2592000000</code>	30-day lookback window

Screenshot 7 — Attack Map KQL Query

Attack Map KQL Query



What this screenshot shows:

The Query Settings view of the attack map workbook — the full KQL query powering the map, with its data source set to **LAW-SOC-lab** and a 30-day time range.

Its role in the lab:

This is the analytical engine behind the attack map. It joins Windows Security Events with the GeoIP watchlist to produce a geographic summary of all brute-force activity detected by the SIEM.

```
let GeoIPDB_FULL = _GetWatchlist("geoip");
let WindowsEvents = SecurityEvent;
WindowsEvents
| where EventID == 4625
| order by TimeGenerated desc
| evaluate ipv4_lookup(GeoIPDB_FULL, IPAddress, network)
| summarize FailureCount = count() by IPAddress, latitude, longitude, city_name, country_name
| project FailureCount,
    AttackerIp = IPAddress,
    latitude,
    longitude,
    city = city_name,
    country = country_name,
    friendly_location = strcat(city_name, " (", country_name, ")")
```

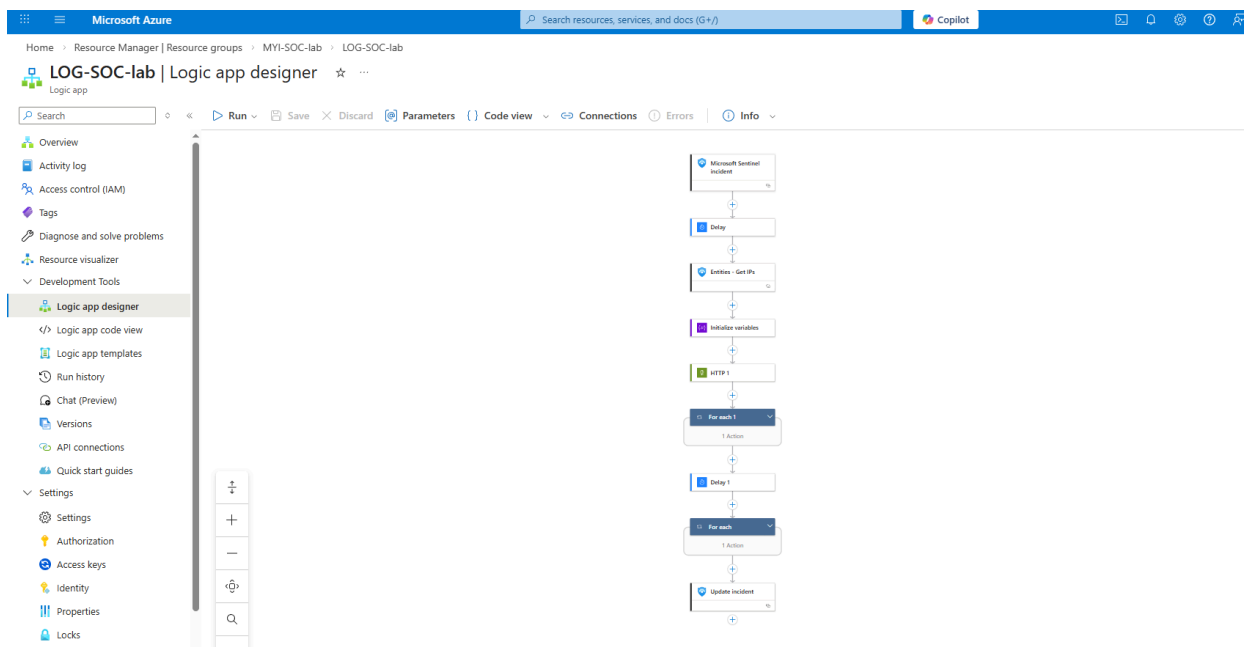
Query breakdown:

Step	Purpose
<code>_GetWatchlist("geoip")</code>	Load the GeoIP CIDR lookup table into memory
<code>where EventID == 4625</code>	Filter to failed logon events only

Step	Purpose
<code>evaluate ipv4_lookup(...)</code>	Match each attacker IP against GeoIP CIDR ranges to get location
<code>summarize FailureCount = count()</code>	Count total failures per IP and location
<code>project</code>	Select and rename only the columns the map needs
<code>strcat(city_name, " (", country_name, ")")</code>	Build a human-readable tooltip label

Screenshot 8 — Logic App Designer Overview

Logic App Designer



What this screenshot shows:

The Azure Logic App Designer canvas for `LOG-SOC-lab` — a visual representation of the entire automated response playbook as a sequential workflow, hosted inside the `MYI-SOC-lab` resource group.

Its role in the lab:

This is the automated response engine. It is the playbook that executes every time

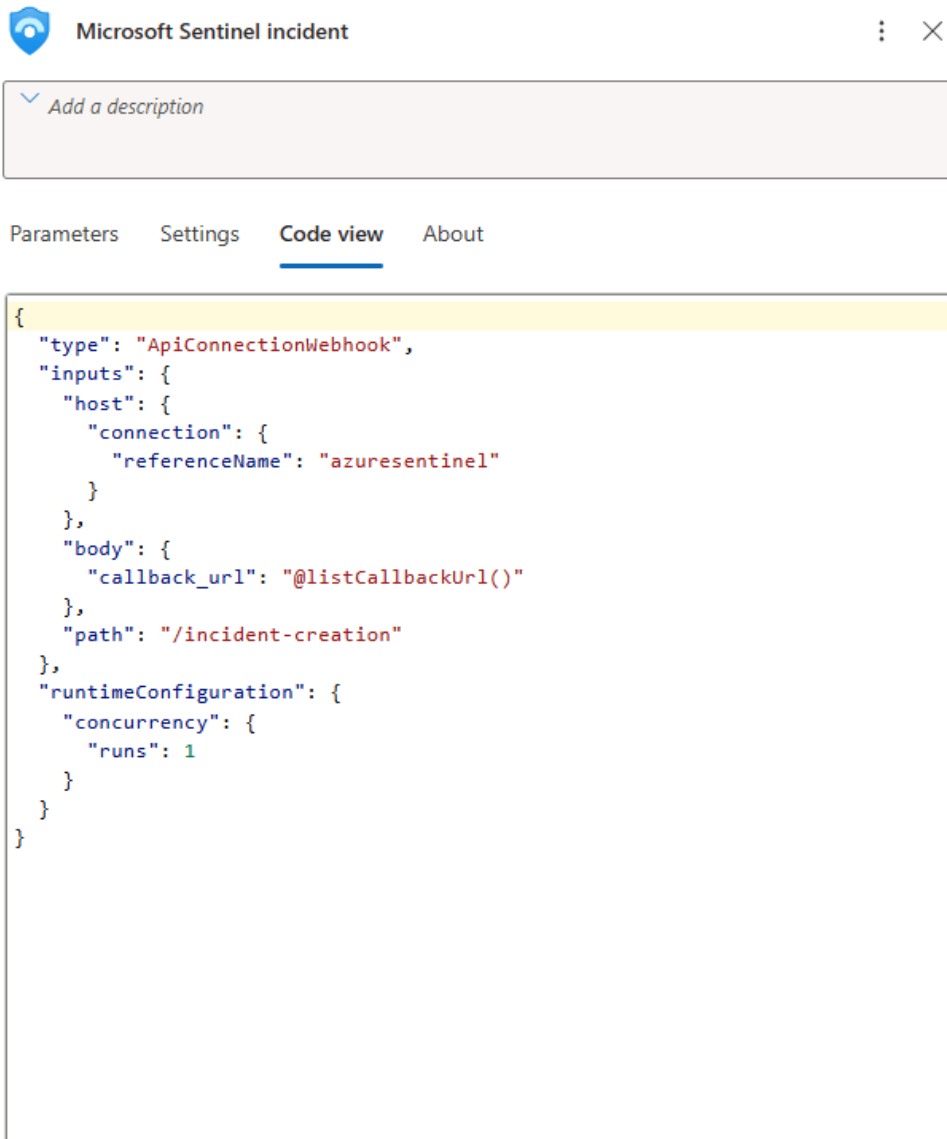
Sentinel creates a brute-force incident, removing the need for a human analyst to manually block attacking IPs. The full workflow is visible as a vertical chain:

```
[Trigger]  Microsoft Sentinel Incident
↓
[Step 1]   Delay – 45 seconds
↓
[Step 2]   Entities - Get IPs
↓
[Step 3]   Initialize Variables
↓
[Step 4]   HTTP 1 – GET NSG Rules
↓
[Step 5]   For each 1 – Find MaxPriority
↓
[Step 6]   Delay 1 – 5 seconds
↓
[Step 7]   For each – Inject deny rules per IP
↓
[Step 8]   Update Incident
```

Each step feeds directly into the next, forming a fully automated detect-to-block pipeline that completes in under 2 minutes from incident creation.

Screenshot 9 — Trigger: Microsoft Sentinel Incident

Sentinel Trigger



What this screenshot shows:

The code view of the Logic App's trigger step — the **Microsoft Sentinel incident** webhook connector that starts the entire playbook execution.

Its role in the lab:

The trigger listens for new incident creation events from Microsoft Sentinel. When the analytics rule detects 10+ failed logons and Sentinel creates an incident, this trigger fires and kicks off the automated response chain.

```
{
  "type": "ApiConnectionWebhook",
```

```

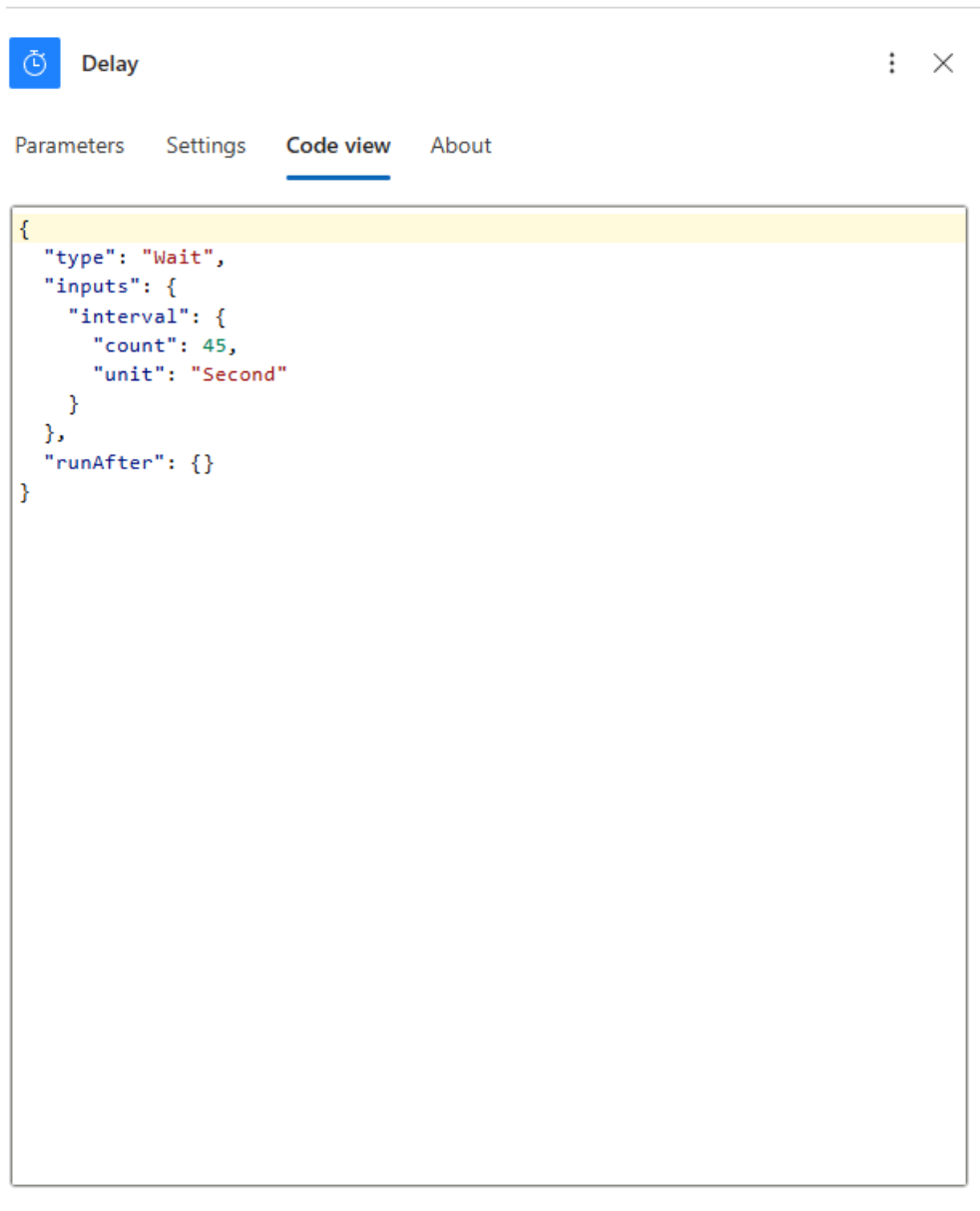
"inputs": {
  "host": {
    "connection": { "referenceName": "azuresentinel" }
  },
  "body": { "callback_url": "@listCallbackUrl()" },
  "path": "/incident-creation"
},
"runtimeConfiguration": {
  "concurrency": { "runs": 1 }
}
}

```

Property	Purpose
<code>path: "/incident-creation"</code>	Listens specifically for incident creation events — not updates or closures
<code>concurrency.runs: 1</code>	Only one playbook instance runs at a time — prevents race conditions when writing NSG rules

Screenshot 10 — Delay: 45 Seconds

Delay 45s



What this screenshot shows:

The code view of the first **Delay** action — a 45-second wait immediately after the trigger fires.

Its role in the lab:

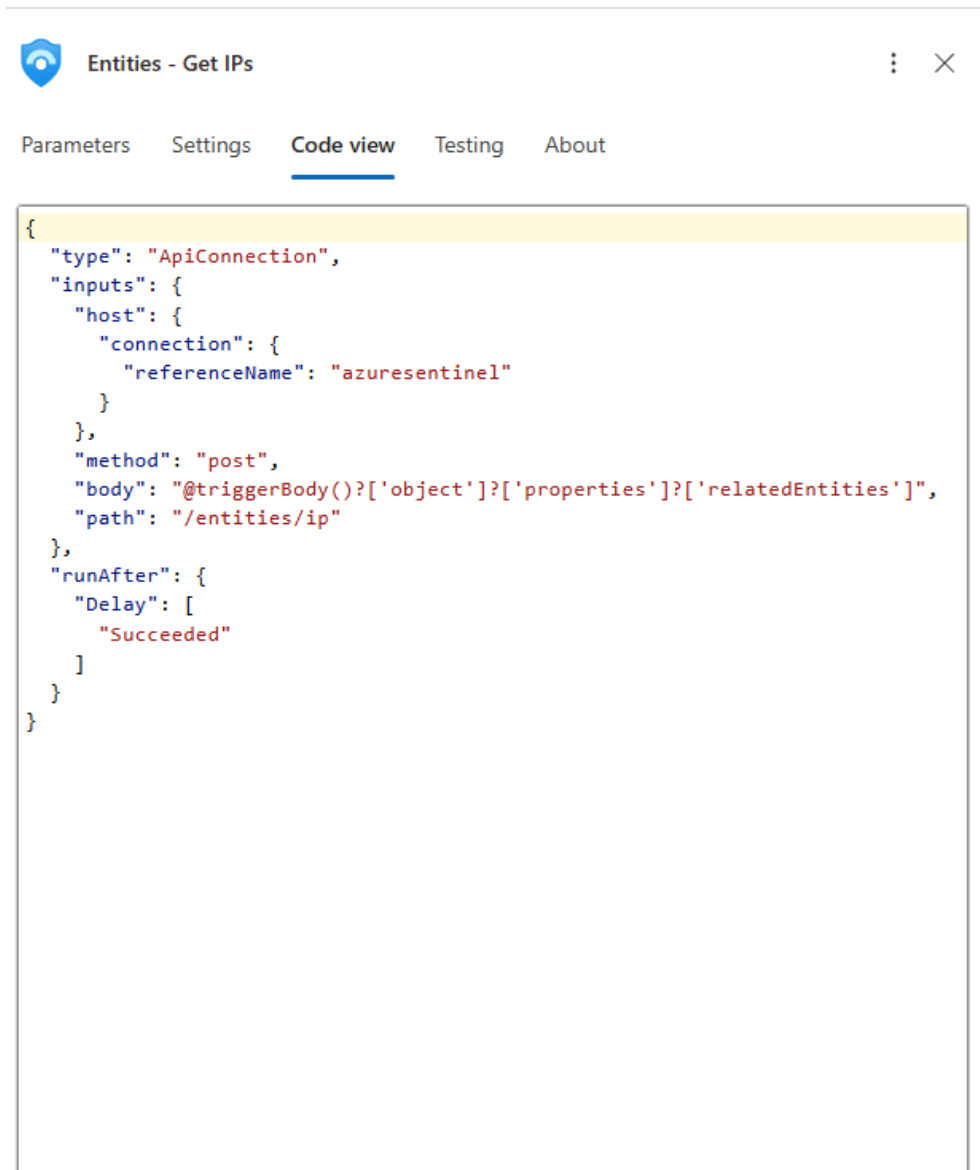
When Sentinel creates an incident, the entity data (IP addresses extracted from the alert) may not be fully committed and available in the API immediately. This 45-second pause ensures all related entities are fully propagated before the next step tries to read them.

```
{
  "type": "Wait",
  "inputs": {
    "interval": {
      "count": 45,
      "unit": "Second"
    }
  },
  "runAfter": {}
}
```

Without this delay, the **Entities - Get IPs** step could execute before the IP entities are available and return an empty list — meaning no IPs would ever be blocked.

Screenshot 11 — Entities: Get IPs

Get IPs



What this screenshot shows:

The code view of the **Entities - Get IPs** action — a Sentinel API connector step that extracts all IP address entities from the incident.

Its role in the lab:

This step retrieves the actual attacker IP addresses from the Sentinel incident so they can be used to create NSG deny rules. It passes the incident's `relatedEntities` list to the Sentinel API and receives back structured IP objects.

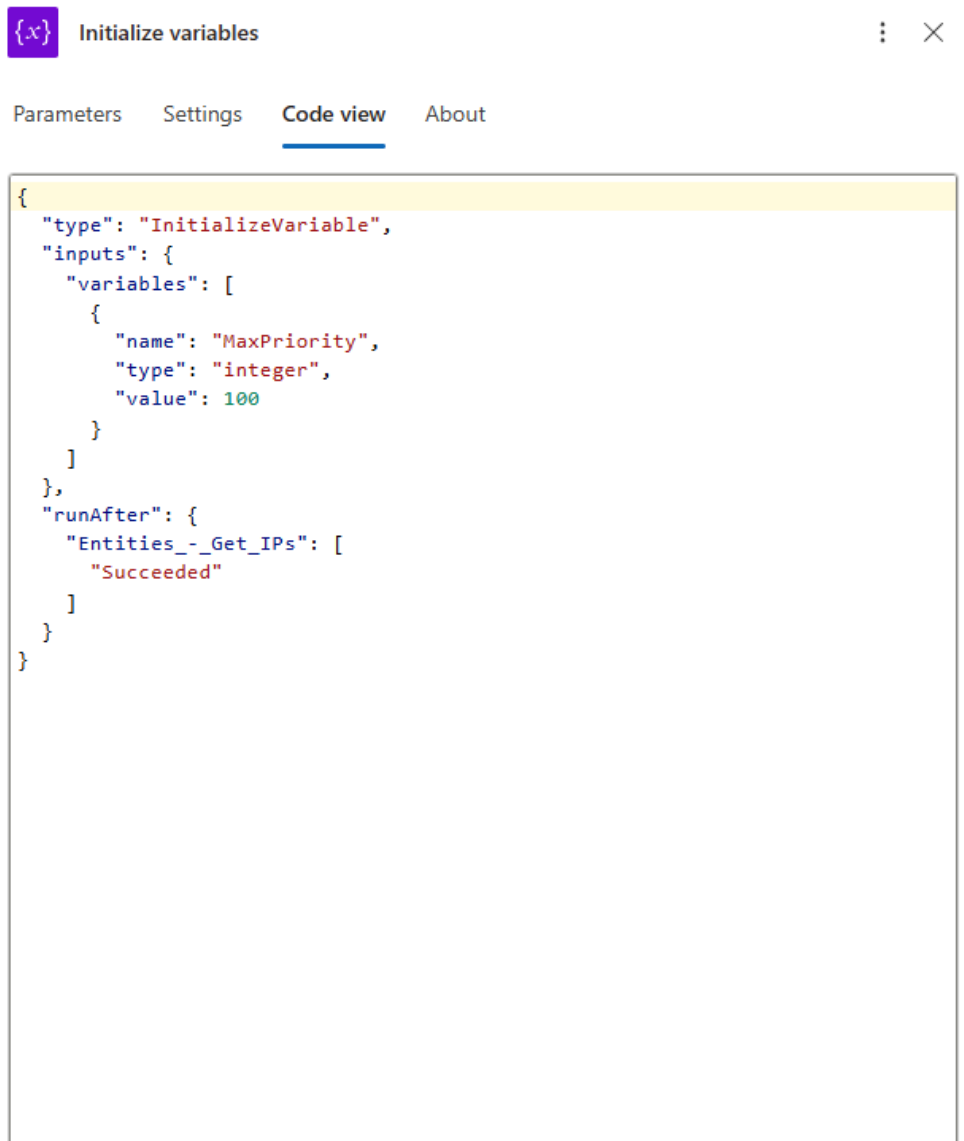
```
{
  "type": "ApiConnection",
  "inputs": {
    "host": {
      "connection": { "referenceName": "azuresentinel" }
    },
    "method": "post",
    "body": "@triggerBody()?['object']?['properties']?['relatedEntities']",
    "path": "/entities/ip"
  },
  "runAfter": {
    "Delay": [ "Succeeded" ]
  }
}
```

Property	Purpose
<code>@triggerBody()?['object']?['properties']?['relatedEntities']</code>	Reads all entities attached to the incident from the trigger payload
<code>path: "/entities/ip"</code>	Filters to IP-type entities only — returns objects with <code>Address</code> fields
<code>runAfter: { "Delay": ["Succeeded"] }</code>	Only executes after the 45-second delay completes successfully

The output of this step — a list of IP objects — is what the final For each loop iterates to build the deny rules.

Screenshot 12 — Initialize Variables

Initialize Variables



The screenshot shows a window titled "Initialize variables" with a purple icon containing a curly brace and an 'x'. Below the title bar are four tabs: "Parameters", "Settings", "Code view" (which is selected and underlined), and "About". The main area displays a JSON configuration for an "InitializeVariable" action. The JSON is as follows:

```
{
  "type": "InitializeVariable",
  "inputs": {
    "variables": [
      {
        "name": "MaxPriority",
        "type": "integer",
        "value": 100
      }
    ]
  },
  "runAfter": {
    "Entities_-_Get_IPs": [
      "Succeeded"
    ]
  }
}
```

What this screenshot shows:

The code view of the **Initialize variables** action, which creates an integer variable called `MaxPriority` with a starting value of `100`.

Its role in the lab:

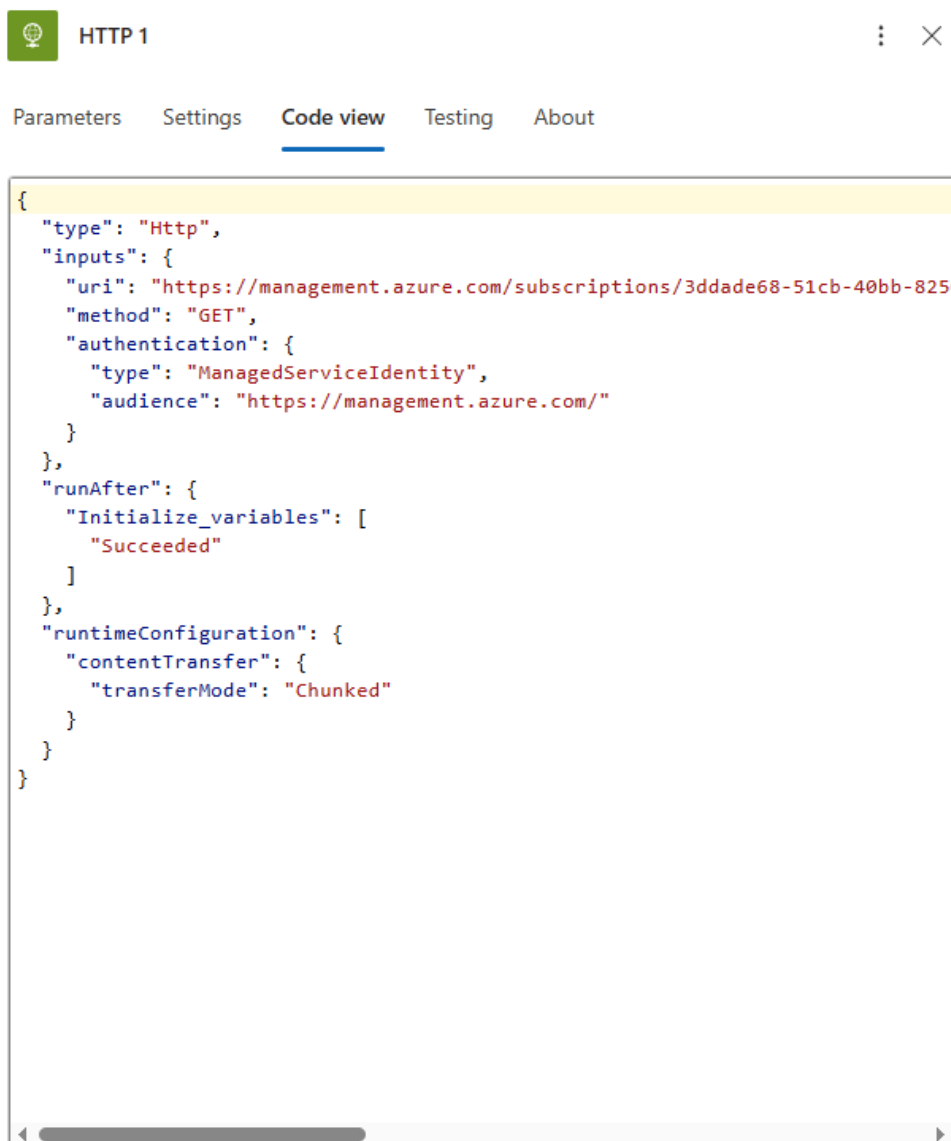
NSG security rules each require a unique priority number. Before inserting a new deny rule, the playbook needs to find the current highest priority number in use so the new rule can be inserted cleanly above it — avoiding conflicts.


```
{
  "type": "InitializeVariable",
  "inputs": {
    "variables": [{
      "name": "MaxPriority",
      "type": "integer",
      "value": 100
    }]
  },
  "runAfter": {
    "Entities_-_Get_IPs": ["Succeeded"]
  }
}
```

`MaxPriority` starts at `100` as the minimum baseline. The subsequent **For each 1** loop then iterates all existing NSG rules and updates this variable to the highest value found. The final deny rules are inserted at `MaxPriority + 1`.

Screenshot 13 — HTTP 1: GET NSG Rules

HTTP 1 GET



The screenshot shows a web interface for an 'HTTP 1' action. At the top, there's a green icon with a globe and the text 'HTTP 1'. Below this is a navigation bar with tabs: 'Parameters', 'Settings', 'Code view' (which is selected and underlined), 'Testing', and 'About'. The main area displays a JSON configuration for the HTTP action. The JSON is as follows:

```
{
  "type": "Http",
  "inputs": {
    "uri": "https://management.azure.com/subscriptions/3ddade68-51cb-40bb-8250...",
    "method": "GET",
    "authentication": {
      "type": "ManagedServiceIdentity",
      "audience": "https://management.azure.com/"
    }
  },
  "runAfter": {
    "Initialize_variables": [
      "Succeeded"
    ]
  },
  "runtimeConfiguration": {
    "contentTransfer": {
      "transferMode": "Chunked"
    }
  }
}
```

What this screenshot shows:

The code view of the **HTTP 1** action — an authenticated REST API call to the Azure Resource Manager to retrieve all current security rules from `CORP-NET-EAST-nsg`.

Its role in the lab:

Before writing new rules, the playbook must read the existing NSG rules to determine what priority numbers are already in use. This prevents collisions where two rules share the same priority, which would cause the Azure API to reject the write request.

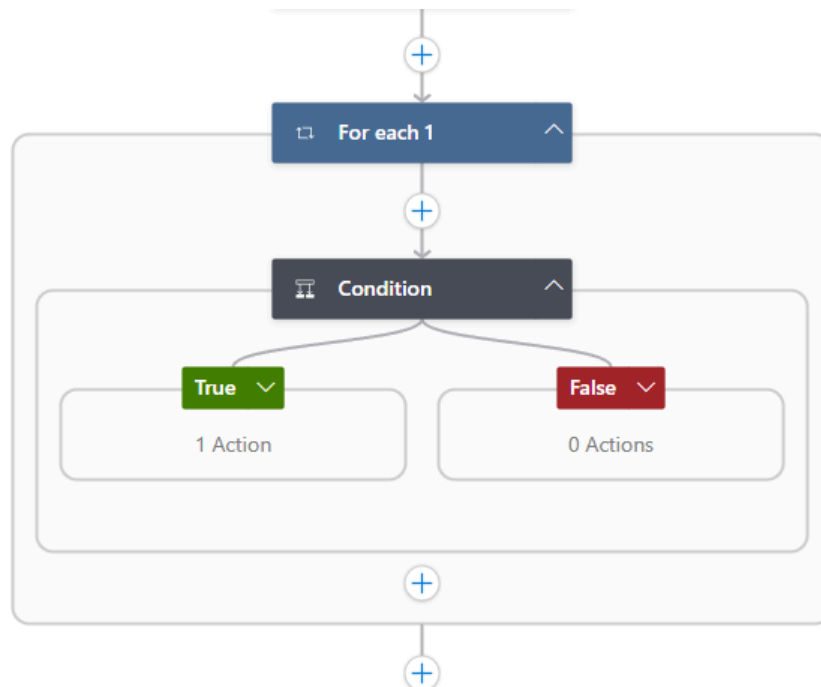
```
{
  "type": "Http",
  "inputs": {
    "uri": "https://management.azure.com/subscriptions/3ddade68-51cb-40bb-8250-.../securityRules",
    "method": "GET",
    "authentication": {
      "type": "ManagedServiceIdentity",
      "audience": "https://management.azure.com/"
    }
  },
  "runtimeConfiguration": {
    "contentTransfer": { "transferMode": "Chunked" }
  }
}
```

Property	Purpose
<code>ManagedServiceIdentity</code>	The Logic App's own Azure identity is used — no stored credentials anywhere
<code>audience:</code> <code>"https://management.azure.com/"</code>	Scopes the access token to the Azure management API
<code>transferMode: "Chunked"</code>	Handles large NSG rule lists without truncation

The response body `@body('HTTP_1')?['properties']?['securityRules']` is what the For each 1 loop iterates in the next step.

Screenshot 14 — For Each 1: Designer View

For each 1 Designer



What this screenshot shows:

The visual designer view of the **For each 1** loop — showing its internal structure: a **Condition** block with a **True** branch (1 Action) and a **False** branch (0 Actions).

Its role in the lab:

This loop iterates every existing NSG rule returned by the HTTP 1 GET call and finds the highest priority number currently in use. It is the “read before write” safety mechanism that ensures new deny rules never conflict with existing ones.

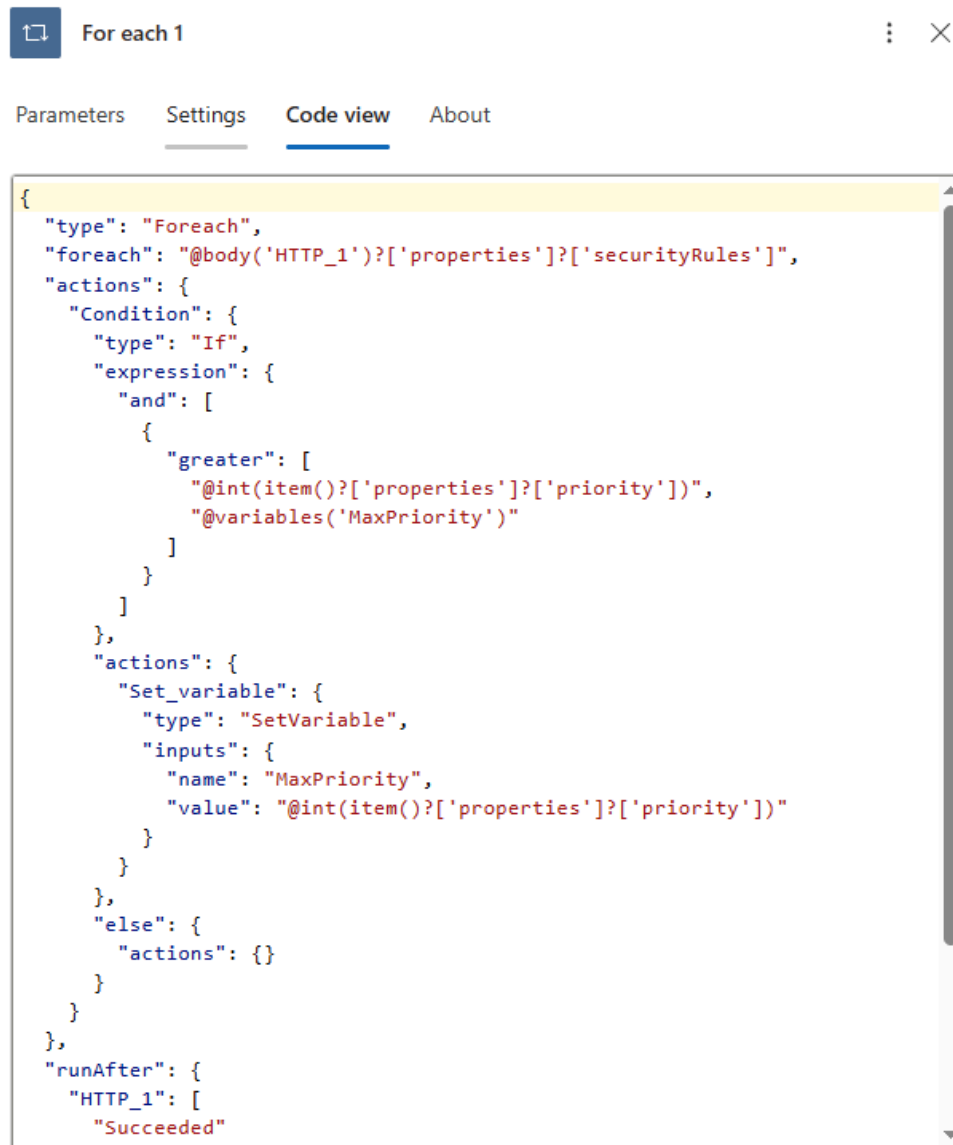
The logic:

- For each existing NSG rule, check: is this rule’s priority **greater than** the current `MaxPriority`?
- **True:** update `MaxPriority` to this rule’s priority value
- **False:** do nothing, move to the next rule

After the loop completes, `MaxPriority` holds the definitive highest priority number assigned in the NSG.

Screenshot 15 — For Each 1: Code View

For each 1 Code View



The screenshot shows the 'For each 1' configuration window in Microsoft Sentinel. The 'Code view' tab is selected, displaying the following JSON code:

```
{
  "type": "Foreach",
  "foreach": "@body('HTTP_1')?['properties']?['securityRules']",
  "actions": {
    "Condition": {
      "type": "If",
      "expression": {
        "and": [
          {
            "greater": [
              "@int(item()?['properties']?['priority'])",
              "@variables('MaxPriority')"
            ]
          }
        ]
      },
      "actions": {
        "Set_variable": {
          "type": "SetVariable",
          "inputs": {
            "name": "MaxPriority",
            "value": "@int(item()?['properties']?['priority'])"
          }
        }
      },
      "else": {
        "actions": {}
      }
    }
  },
  "runAfter": {
    "HTTP_1": [
      "Succeeded"
    ]
  }
}
```

What this screenshot shows:

The full JSON code view of the **For each 1** loop — the complete logic including the foreach data source, the condition expression, and the set variable action in one view.

Its role in the lab:

This is the technical implementation of the priority-detection algorithm:

```
{
  "type": "Foreach",
  "foreach": "@body('HTTP_1')?['properties']?['securityRule"
```

```

s']",
  "actions": {
    "Condition": {
      "type": "If",
      "expression": {
        "and": [{
          "greater": [
            "@int(item()?['properties']['priority'])",
            "@variables('MaxPriority')"
          ]
        }]
      },
      "actions": {
        "Set_variable": {
          "type": "SetVariable",
          "inputs": {
            "name": "MaxPriority",
            "value": "@int(item()?['properties']['priority'])"
          }
        }
      }
    },
    "else": { "actions": {} }
  }
}

```

Expression	Purpose
@body('HTTP_1')?['properties']['securityRules']	The array of existing NSG rules to loop over
@int(item()?['properties']['priority'])	Reads each rule's priority as an integer
greater(..., @variables('MaxPriority'))	Compares it to the current maximum
Set_variable	Overwrites MaxPriority when a higher value is found

Screenshot 16 — Condition Inside For Each 1



Condition Code View

What this screenshot shows:

The isolated code view of the **Condition** action inside the For each 1 loop — the If/Else logic and the Set variable action shown separately from the loop container.

Its role in the lab:

This is the core comparison logic of the priority-finding algorithm. Each rule in the NSG is evaluated against this condition:

```

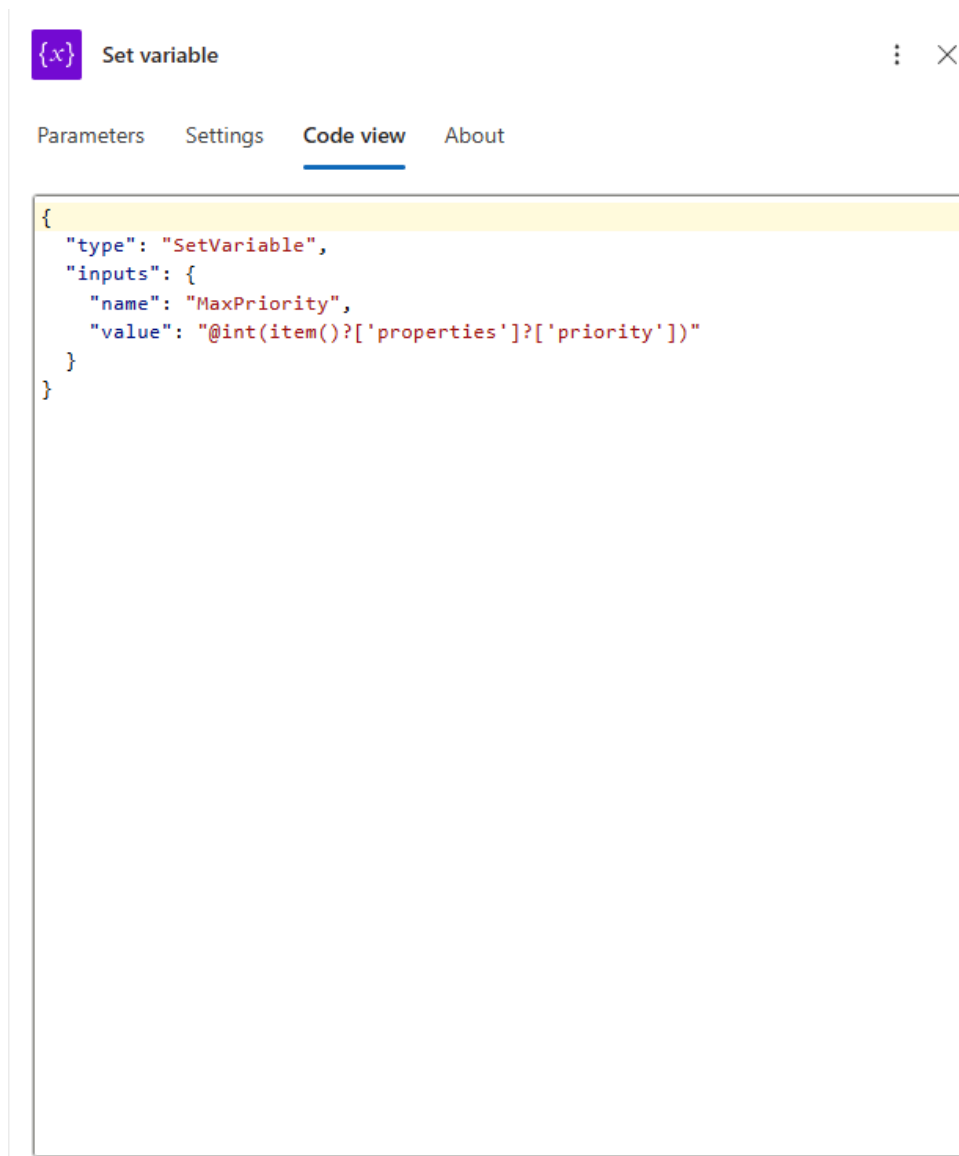
{
  "type": "If",
  "expression": {
    "and": [{
      "greater": [
        "@int(item()?['properties']['priority'])",
        "@variables('MaxPriority')"
      ]
    }]
  },
  "actions": {
    "Set_variable": {
      "type": "SetVariable",
      "inputs": {
        "name": "MaxPriority",
        "value": "@int(item()?['properties']['priority'])"
      }
    }
  },
  "else": { "actions": {} }
}

```

The `else` branch is intentionally empty — if a rule's priority is not greater than the current maximum, no action is needed. The loop simply moves to the next rule.

Screenshot 17 — True Branch: Set Variable

Set Variable



What this screenshot shows:

The isolated code view of the **Set variable** action inside the True branch — the action that executes when a higher-priority NSG rule is found during the loop.

Its role in the lab:

This is the update step that keeps `MaxPriority` in sync with the highest rule priority found so far:

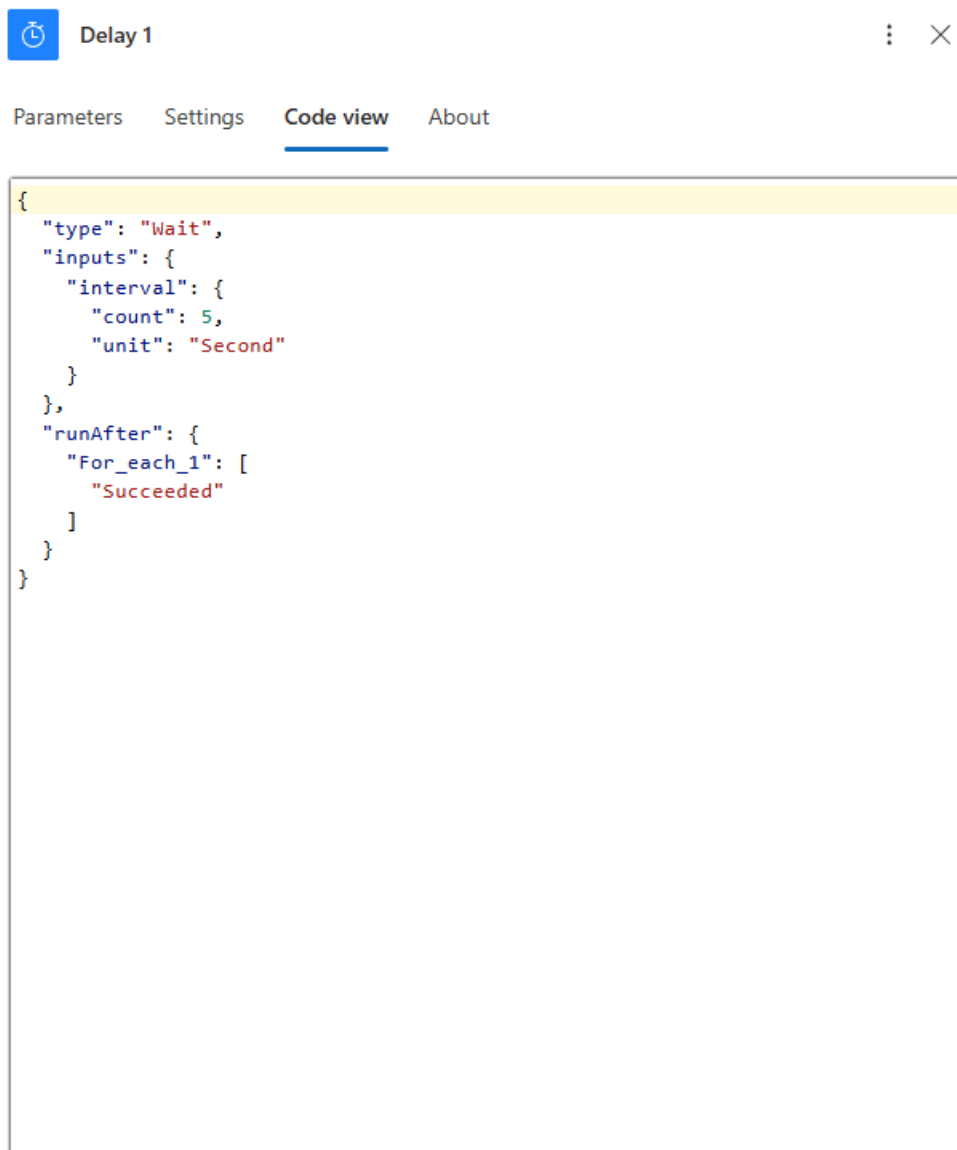
```
{
  "type": "SetVariable",
  "inputs": {
```

```
"name": "MaxPriority",  
"value": "@int(item()?['properties']['priority'])"  
}  
}
```

Each time a rule with a higher priority is found, this action overwrites `MaxPriority` with the new value. By the time the loop finishes, `MaxPriority` is guaranteed to be the absolute highest priority currently assigned in the NSG — so the new deny rule can safely be placed at `MaxPriority + 1`.

Screenshot 18 — Delay 1: 5 Seconds

Delay 1



What this screenshot shows:

The code view of **Delay 1** — a 5-second wait between the For each 1 loop (read phase) and the For each 2 loop (write phase).

Its role in the lab:

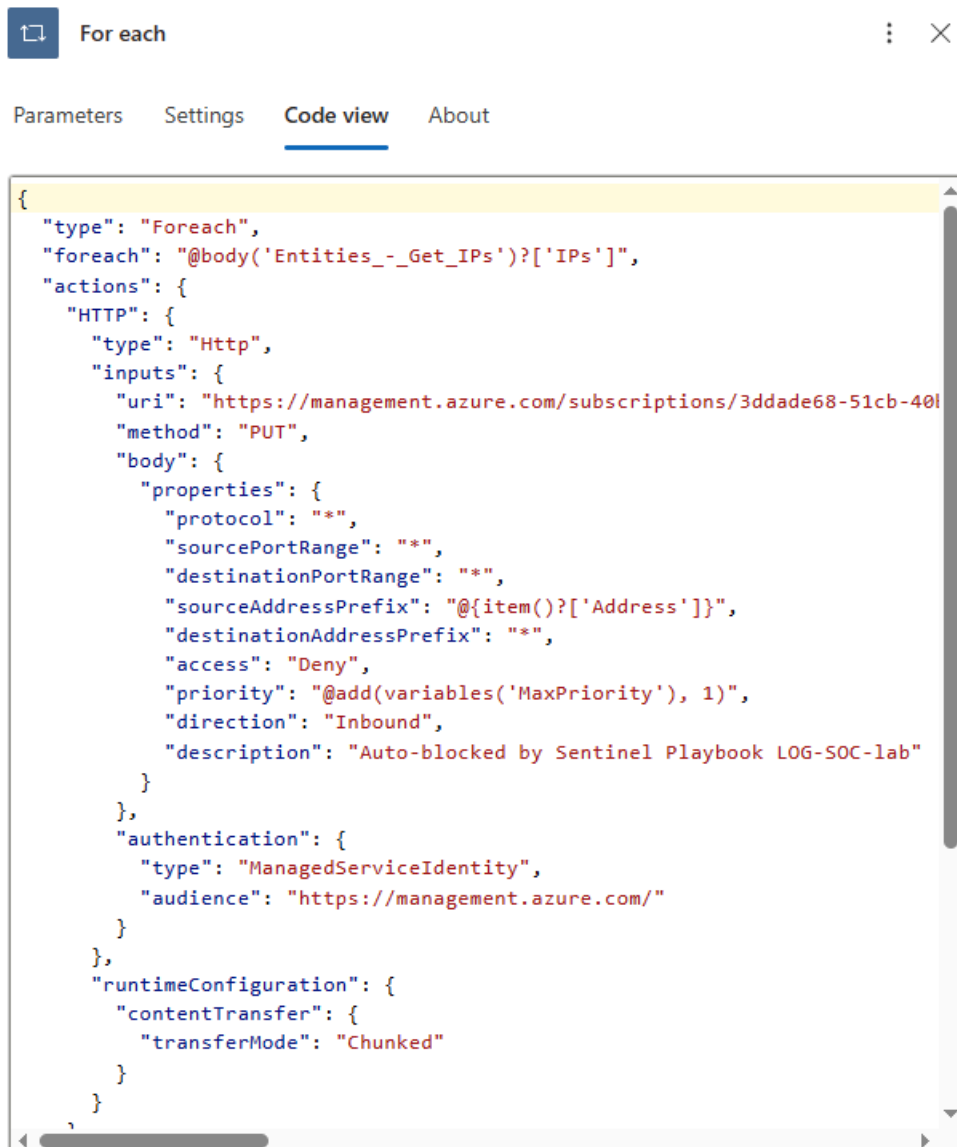
This short pause acts as a rate-limit buffer between the read and write phases. Azure's Resource Manager API has rate limits on write operations — sending multiple PUT requests immediately after a GET can trigger throttling errors (HTTP 429). The 5-second pause gives the API time to process before write operations begin.

```
{
  "type": "Wait",
  "inputs": {
    "interval": {
      "count": 5,
      "unit": "Second"
    }
  },
  "runAfter": {
    "For_each_1": ["Succeeded"]
  }
}
```

`runAfter: { "For_each_1": ["Succeeded"] }` ensures this step only runs after the priority-finding loop has completed successfully.

Screenshot 19 — For Each 2: Inject Deny Rules

For each 2 Code View



```
{
  "type": "Foreach",
  "foreach": "@body('Entities_-_Get_IPs')['IPs']",
  "actions": {
    "HTTP": {
      "type": "Http",
      "inputs": {
        "uri": "https://management.azure.com/subscriptions/3ddade68-51cb-401",
        "method": "PUT",
        "body": {
          "properties": {
            "protocol": "*",
            "sourcePortRange": "*",
            "destinationPortRange": "*",
            "sourceAddressPrefix": "@{item()}['Address']]",
            "destinationAddressPrefix": "*",
            "access": "Deny",
            "priority": "@add(variables('MaxPriority'), 1)",
            "direction": "Inbound",
            "description": "Auto-blocked by Sentinel Playbook LOG-SOC-lab"
          }
        },
        "authentication": {
          "type": "ManagedServiceIdentity",
          "audience": "https://management.azure.com/"
        },
        "runtimeConfiguration": {
          "contentTransfer": {
            "transferMode": "Chunked"
          }
        }
      }
    }
  }
}
```

What this screenshot shows:

The code view of the second **For each** loop — the action that iterates over all attacker IPs from the Sentinel incident and issues an API call to block each one in the NSG.

Its role in the lab:

This is the enforcement step — it is where the actual blocking happens. For every IP address attached to the incident, it sends a PUT request to the Azure NSG REST API creating a new inbound Deny rule targeting that specific IP.

```

{
  "type": "Foreach",
  "foreach": "@body('Entities_-_Get_IPs')?['IPs']",
  "actions": {
    "HTTP": {
      "type": "Http",
      "inputs": {
        "uri": "https://management.azure.com/.../CORP-NET-EAS
T-nsg/securityRules/...",
        "method": "PUT",
        "body": {
          "properties": {
            "protocol": "*",
            "sourcePortRange": "*",
            "destinationPortRange": "*",
            "sourceAddressPrefix": "@{item()?['Address']}",
            "destinationAddressPrefix": "*",
            "access": "Deny",
            "priority": "@add(variables('MaxPriority'), 1)",
            "direction": "Inbound",
            "description": "Auto-blocked by Sentinel Playbook
LOG-SOC-lab"
          }
        },
        "authentication": {
          "type": "ManagedServiceIdentity",
          "audience": "https://management.azure.com/"
        }
      }
    }
  }
}

```

Property	Purpose
<code>foreach: "@body('Entities_-_Get_IPs')?['IPs']"</code>	Iterates the list of attacker IPs from the Sentinel incident
<code>sourceAddressPrefix: "@{item()?['Address']}"</code>	Dynamically inserts each attacker IP into the deny rule
<code>priority: "@add(variables('MaxPriority'), 1)"</code>	Places the new rule just above all existing rules — no conflicts
<code>description</code>	Audit trail — every auto-generated rule is clearly labelled
<code>ManagedServiceIdentity</code>	No credentials stored — the Logic App's Azure identity handles auth

Screenshot 20 — HTTP 2: NSG Deny Rule Parameters

HTTP 2 Parameter

HTTP

Parameters Settings Code view Testing About

URI *

https://management.azure.com/subscriptions/3ddade68-51cb-40bb-8250-5fd3b1e6c321/resourceGroups/MYI-SOC-lab/providers/Microsoft.Network/networkSecurityGroups/CORP-NET-EAST-nsg/securityRules/ *concat(...)* ?api-version=2023-05-01

Method *

PUT

Headers

Enter key Enter value

Queries

Enter key Enter value

Body

```
{
  "properties": {
    "protocol": "*",
    "sourcePortRange": "*",
    "destinationPortRange": "*",
    "sourceAddressPrefix": "IPs Address",
    "destinationAddressPrefix": "*",
    "access": "Deny",
    "priority": add(...),
    "direction": "Inbound",
    "description": "Auto-blocked by Sentinel Playbook LOG-SOC-lab"
  }
}
```

Cookie

What this screenshot shows:

The **Parameters** tab view of the HTTP action inside the For each 2 loop — the full NSG REST API PUT call shown in a readable form, with the target URI, HTTP method, and complete request body visible.

Its role in the lab:

This view makes the IP blocking action explicit and auditable. It shows exactly which Azure resource is being modified, with what payload, and how each attacker IP becomes a named security rule in the NSG.

Target URI:


```
https://management.azure.com/subscriptions/3ddade68-51cb-40b.....  
/resourceGroups/MYI-SOC-lab  
/providers/Microsoft.Network/networkSecurityGroups/CORP-NET-EAST-nsg  
/securityRules/<rule-name>  
?api-version=2023-05-01
```

Request body:

```
{  
  "properties": {  
    "protocol": "*",  
    "sourcePortRange": "*",  
    "destinationPortRange": "*",  
    "sourceAddressPrefix": "<Attacker IP>",  
    "destinationAddressPrefix": "*",  
    "access": "Deny",  
    "priority": "<MaxPriority + 1>",  
    "direction": "Inbound",  
    "description": "Auto-blocked by Sentinel Playbook LOG-SOC  
-lab"  
  }  
}
```

Design decisions:

Decision	Rationale
<code>method: PUT</code>	Creates or updates a named rule — idempotent and safe to re-run
<code>protocol: "*"</code>	Blocks the IP across all protocols — TCP, UDP, ICMP
<code>sourceAddressPrefix = attacker IP</code>	Surgical block — only the offending IP is denied
<code>destinationPortRange: "*"</code>	Blocks the IP on all ports — prevents pivoting to other services
<code>priority = MaxPriority + 1</code>	Safe insertion above all existing rules — no overwrites
<code>direction: Inbound</code>	Blocks incoming traffic from the attacker only

Decision	Rationale
<code>access: Deny</code>	Traffic is dropped
<code>api-version: 2023-05-01</code>	Current stable Azure Network REST API version
MSI authentication	Zero stored credentials — identity scoped to the Logic App only

Screenshot 21 — Incident update

update incident


Update incident
⋮
✕

Parameters
Settings
Code view
Testing
About

Incident ARM id *


Incident ARM ID
✕

Tags to add

+ Add new item


Assign/Unassign owner

No change
✕

Owner Object Id / UPN

Unique identifier of a user (Ex. 'user@tenant.onmicrosoft.com' or '5f6ce5c7-b521...

Severity

Incident severity
▼

Status

Closed
▼

* Classification reason

TruePositive - SuspiciousActivity
▼

Close reason text

Attacker IP has been automatically blocked in the NSG by Logic App SOC-lab and the incident is now resolved

Advanced parameters

What this screenshot shows:

The **Parameters** tab of the **Update incident** action — the final step of the Logic App playbook. This Sentinel connector action closes the incident and writes a resolution comment back into Sentinel once the IP blocking is complete.

Its role in the lab:

This step closes the loop on the automated response. After the attacker IP has been injected as a deny rule into the NSG, there is no longer an active threat — the incident can be resolved automatically. This keeps the Sentinel incident queue clean and provides a clear audit trail of what happened and why the incident was closed.

Configuration:

Parameter	Value
Incident ARM ID	<code>Incident ARM ID</code> (dynamic — pulled from the trigger payload)
Assign/Unassign owner	No change
Severity	Incident severity (no change)
Status	Closed
Classification reason	TruePositive - SuspiciousActivity
Close reason text	<code>Attacker IP has been automatically blocked in the NSG by Logic App SOC-lab and the incident is now resolved</code>

Key design decisions:

- **Status: Closed** — the incident is fully resolved by the automation, no analyst action needed
- **TruePositive - SuspiciousActivity** — the classification is accurate: a real brute-force attempt was detected and blocked, not a false alarm
- **Close reason text** — provides a human-readable audit trail inside the incident, so any analyst reviewing it later understands exactly what happened and which system handled it

Screenshot 22 — Sentinel Incidents Queue

Incidents & alerts	Brute Force - Auto Block IP	11	(22)	High	Credential access	0/7
Incidents	Brute Force - Auto Block IP	18	(22)	High	Credential access	0/5
Alerts	Brute Force - Auto Block IP	3	(22)	High	Credential access	0/6
Hunting	Brute Force - Auto Block IP	33	(21)	High	Credential access	0/3
Actions & submissions	Brute Force - Auto Block IP	28	(21)	High	Credential access	0/2
Partner catalog	Brute Force - Auto Block IP	23	(21)	High	Credential access	0/2
Threat intelligence	Brute Force - Auto Block IP	9	(21)	High	Credential access	0/2
Assets	Brute Force - Auto Block IP	1	(21)	High	Credential access	0/2
Microsoft Sentinel	Brute Force - Auto Block IP	32	(19)	High	Credential access	0/1
Search	Brute Force - Auto Block IP	31	(19)	High	Credential access	0/1
Threat management	Brute Force - Auto Block IP	30	(19)	High	Credential access	0/1
Content management	Brute Force - Auto Block IP	29	(19)	High	Credential access	0/1
Configuration	Brute Force - Auto Block IP	27	(19)	High	Credential access	0/1
Endpoints	Brute Force - Auto Block IP	25	(19)	High	Credential access	0/1

What this screenshot shows:

The Microsoft Sentinel **Incidents** queue showing a long list of triggered **"Brute Force - Auto Block IP"** incidents, all classified as **High** severity under the **Credential Access** tactic.

Its role in the lab:

This screenshot is the proof that the analytics rule and the automated pipeline are working end-to-end in real conditions. Each row in this list represents a real brute-force event that was:

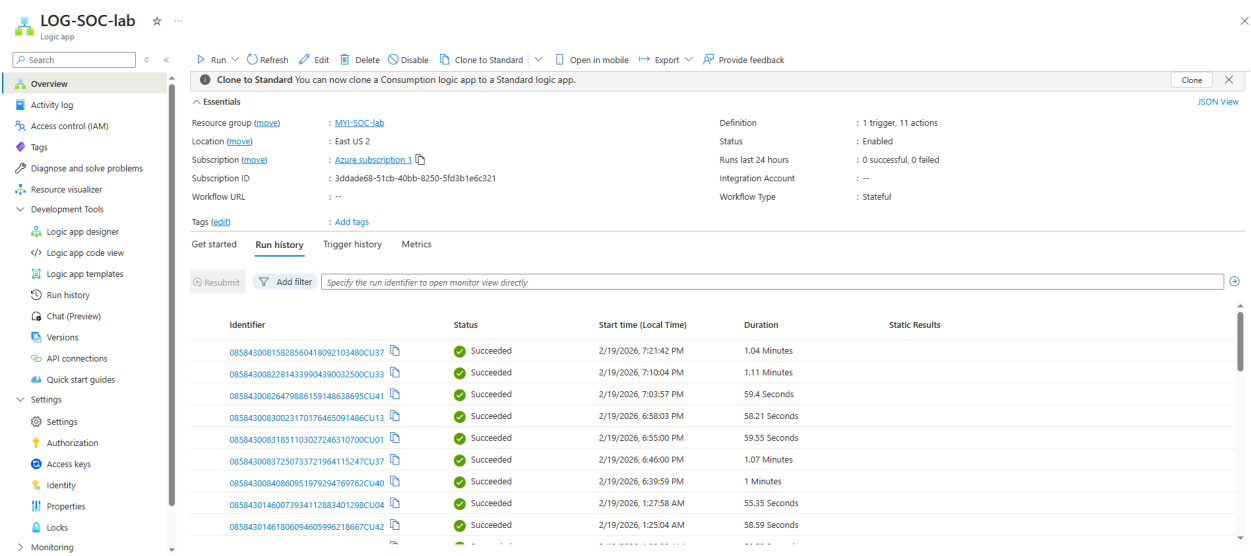
1. Detected by the analytics rule (≥ 10 failed logons / 5 min from a single IP)
2. Escalated into a Sentinel incident
3. Processed by the Logic App playbook
4. Resolved with the attacker IP blocked in the NSG

Key details visible:

Column	Meaning
Name	Brute Force - Auto Block IP — all generated by the same analytics rule
Alert count (circled number)	Number of individual alerts grouped into each incident — values of 19–22 show sustained attack waves
Severity	High across all incidents
Tactic	Credential Access — consistent with T1110 brute force
Comments (0/x)	Incident comment count — confirms the Update Incident step ran

The volume of incidents (33+ visible) over a short period reflects the intensity of real-world automated RDP scanning activity against the exposed VM, and demonstrates that the detection and response pipeline scaled to handle repeated attack waves without any manual intervention.

Screenshot 23 — Logic App Run History



What this screenshot shows:

The **Run history** tab of the **LOG-SOC-lab** Logic App Overview page, showing a list of recent playbook executions with their status, start time, and duration.

Its role in the lab:

This is the operational health dashboard for the automated response system. It confirms the Logic App executed successfully for every Sentinel incident triggered by the analytics rule. Every row in this list corresponds to one complete detect-to-block cycle.

Key details visible:

Field	Value
Resource group	MYI-SOC-lab
Location	East US 2
Definition	1 trigger, 11 actions

Field	Value
Status	Enabled
Workflow type	Stateful — full execution history is retained
Runs last 24 hours	0 successful, 0 failed (at time of overview capture)

All runs completed with **Succeeded** status. Execution time consistently falls between **55 seconds and 1.11 minutes** — meaning from the moment a Sentinel incident is created, the attacker's IP is blocked in the NSG in under 2 minutes, fully automatically.

The **Stateful** workflow type means every step of every run is logged and inspectable — critical for audit trails in a real SOC environment.

Screenshot 24 — NSG: Auto-Injected Deny Rules

The screenshot displays the Azure portal interface for the 'CORP-NET-EAST-nsg' Network Security Group. The 'Overview' tab is selected, showing a list of inbound security rules. The rules are categorized into 'Inbound Security Rules' and 'Outbound Security Rules'. The 'Inbound Security Rules' list includes several rules with a 'DANGER' icon, indicating they are auto-injected deny rules. These rules have names like 'Block-185.156.73.169', 'Block-185.156.73.74', 'Block-196.237.82.55', 'Block-41.225.82.195', 'Block-194.3.181.177', 'Block-46.193.67.244', and 'Block-78.128.112.114'. They all have a priority of 100-304, protocol of 'Any', and action of 'Deny'. The 'Outbound Security Rules' list includes rules like 'AllowVnetInBound', 'AllowAzureLoadBalancerInBound', 'DenyAllInBound', 'AllowVnetOutBound', 'AllowInternetOutBound', and 'DenyAllOutBound'.

What this screenshot shows:

The **CORP-NET-EAST-nsg** Network Security Group inbound rules page showing a mix of the original lab rules and multiple auto-injected deny rules created by the Logic App playbook.

Its role in the lab:

This is the final proof of the entire automated response pipeline working correctly. The NSG rules visible here are the direct output of the Logic App executing —

every `Block-*` rule was created automatically by the playbook in response to a Sentinel brute-force incident, with no human touching the NSG.

Conclusion

What We Built

This lab delivered a fully operational, cloud-native Security Operations Center on Microsoft Azure — built from scratch using real infrastructure, real attack traffic, and real automated response. The environment covered every layer of a modern SOC pipeline:

- A **deliberately exposed Windows Server VM** that attracted genuine internet-facing brute-force attacks within hours of deployment
- A **Log Analytics workspace + Microsoft Sentinel** setup that ingested, stored, and queried all Windows Security Events using KQL
- A **GeoIP-enriched attack map workbook** that visualised the global origin of attacks in real time
- A **custom scheduled analytics rule** that detected brute-force patterns (≥ 10 failed logons / 5 min) and escalated them as High-severity incidents mapped to MITRE ATT&CK T1110
- A **Logic App playbook** that responded to every incident automatically — extracting attacker IPs, computing safe NSG priority placement, injecting deny rules via the Azure REST API, and closing the incident with a written audit trail — all in under 2 minutes, with zero human input

What the Numbers Showed

The lab produced real, unscripted results that validated every component of the pipeline:

Metric	Result
Attacker IPs blocked	7 unique IPs auto-denied in NSG
Sentinel incidents generated	33+ High severity incidents
Top attack volume	39,400 failed logons from a single source

Metric	Result
Human interventions required	0

These are not simulated numbers. The 39,400 failed RDP attempts from Jordanow, Poland represent a real botnet that found the exposed VM and hammered it relentlessly — and the pipeline detected, responded to, and blocked it without any manual action.

What We Learned

1. Cloud infrastructure exposure is immediate.

Within hours of deploying a Windows VM with an open NSG rule, automated scanners found it and began brute-forcing RDP. This is the reality of internet-facing infrastructure — threat actors operate bots that continuously scan the entire IPv4 address space. Security controls must be in place before exposure, not after.

2. SIEM is only useful when data flows through it correctly.

The pipeline only worked because every layer was configured properly — the AMA agent, the DCR, the Log Analytics workspace, the Sentinel connector, and the entity mapping in the analytics rule. A single misconfiguration anywhere in that chain (wrong event scope, missing entity mapping, wrong concurrency setting) would have silently broken the response. Understanding the full data flow is as important as knowing the tools.

3. KQL is the core skill of cloud-native security operations.

Every detection in this lab depended on KQL — from basic pipeline verification queries, to the brute-force detection logic, to the GeoIP-enriched attack map. Proficiency in KQL directly translates to the ability to build detections, investigate incidents, and extract actionable intelligence from raw event data.

4. Automated response must be designed carefully.

The Logic App playbook had several non-obvious design requirements that would have caused failures without careful thought — the 45-second entity propagation delay, the MaxPriority computation loop to avoid NSG rule conflicts, the 5-second rate-limit buffer before write operations, and the concurrency limit of 1 on the trigger. Automation that is rushed or under-tested can fail silently, block legitimate IPs, or create conflicting rules. Good SOAR design requires the same rigour as good code.

5. Managed Identity is the right authentication model for cloud automation.

Using MSI instead of stored credentials in the Logic App means there are no API keys or passwords to rotate, leak, or manage. The Logic App's identity is scoped only to what it needs — the NSG. This is the principle of least privilege applied to automation, and it should be the default approach for any cloud-based playbook.

6. Visibility and response are not the same thing.

The attack map workbook provided excellent situational awareness — but awareness alone does not stop attacks. The combination of visualisation (workbook) + detection (analytics rule) + enforcement (Logic App + NSG) is what makes the pipeline effective. Each layer has a distinct role, and all three are required for a complete response capability.

Skills Demonstrated

Skill Area	Specifics
Azure Infrastructure	Resource groups, VNets, VMs, NSGs, public IPs, managed disks
Microsoft Sentinel	Workspace setup, data connectors, analytics rules, entity mapping, automation rules, workbooks, watchlists
Log Analytics / KQL	Event ingestion, pipeline verification, field projection, ipv4_lookup, GeoIP enrichment, threat-focused filtering
Azure Logic Apps	Webhook triggers, API connectors, HTTP actions, loops, conditions, variables, MSI authentication, Azure REST API
MITRE ATT&CK	T1110 Brute Force — detection, classification, and automated response
Security Operations	Incident lifecycle management, automated triage, audit trails, SOAR playbook design

Final Thought

This lab is a practical demonstration that a solo student with a free Azure trial and a Sentinel trial can build a SOC pipeline that detects and responds to real attacks automatically. The same architecture — analytics rules, Logic App playbooks, NSG enforcement — scales directly to enterprise environments. The concepts, the KQL patterns, the playbook design, and the API-based enforcement model are all production-applicable skills.